

Wichtige Funktionen bei Datenanalysen mit RStudio - Hilfe bei der empirischen Auswertung echter Projekte

Christine Dauth

2023-08-30

1. Installieren und Laden von Paketen

In dieser Veranstaltung geht es darum, Daten selbst zu erheben und diese im Anschluss daran auch auszuwerten. Ein gutes Tool dafür ist R mit dem Interface RStudio. Wir wollen uns nun gemeinsam vertraut machen mit der Funktionsweise von R und den wichtigsten Funktionen, die wir für die Auswertung von Daten brauchen. Dazu nutzen wir verschiedene Übungsdatensätze, die im Internet für Lehrzwecke frei verfügbar gemacht wurden.

Wir wissen bereits, dass R nur sehr grundlegende Funktionen hat. Je umfangreicher wir dieses Tool nutzen möchten, desto mehr Pakete müssen wir laden. Pakete sind vergleichbar mit den Apps eines Smartphones. Einmalig müssen wir die Pakete, die wir benötigen laden. Dies passiert mit der Funktion `install.packages()`. Für die Analyse folgender Daten, benötigen wir folgende Pakete:

```
install.packages("tidyverse")
install.packages("readxl")
install.packages("knitr")
install.packages("skimr")
install.packages("tidyverse")      # für Beispieldatensätze
install.packages("stargazer")     # für deskriptive Statistiken
install.packages("janitor")       # für tabyl-Funktion
install.packages("openxlsx")     # für output in Excel-Dokument
```

Wie gesagt, die Pakete müssen nur ein einziges Mal auf dem PC installiert werden. Damit wir sie aber nutzen können, müssen wir sie jetzt noch laden. Diesen Schritt müssen wir jedes mal, wenn wir RStudio neu öffnen, ausführen. Wir machen dies mit der Funktion `library()`.

```
library(tidyverse)
library(readxl)
library(knitr)
library(skimr)
library(nycflights13)
library(stargazer)
library(janitor)
library(openxlsx)
```

Im nächsten Schritt wollen wir nun auch unser Arbeitsverzeichnis in dem Objekt `pfad` festlegen, damit R immer genau weiß, wo unsere Daten liegen und wo unser Output abgespeichert werden soll. Sollte sich der Verzeichnispfad ändern, dann müssen wir dies nur einmal an genau dieser Stelle tun. Das erspart viel Arbeit!

Das Arbeitsverzeichnis (working directory) ist auf jedem PC und für jeden Studierenden individuell. Das heißt, jeder von Ihnen muss an dieser Stelle seinen ganz eigenen Pfad zu seinem individuellen Arbeitsordner einfügen. Bitte beachten Sie dabei, dass Sie hier die umgekehrten Schrägstriche durch normale Schrägstriche ersetzen müssen.

```
pfad <- "C:/Users/christine.dauth/Documents/4_Empirische Wirtschaftsforschung/Einführung in R"
graf <- "C:/Users/christine.dauth/Documents/4_Empirische Wirtschaftsforschung/Einführung in R/Auswert
ungen/"

setwd(pfad)
```

So, jetzt können wir mit der Analyse unserer Daten beginnen!

2. Daten importieren

Der allererste Schritt besteht nun darin, unsere csv-Daten erstmal einzulesen, man sagt "importieren". Dies geht am einfachsten, wenn wir über das Menü gehen: Wir klicken mit der Maus

File --> Import Data Set --> From Text (readr) . Wie Sie sehen, müssen wir wissen, in welchem Format die Daten vorliegen. In unserem Fall liegen die Daten im csv-Format vor. R kann aber auch viele andere Formate einlesen. Nach dem Importieren zeigt uns die Konsole die zugehörige Funktion. Wir können Sie nun in unser R Skript einfügen, damit wir uns das nächste Mal nicht noch einmal durch das Menü klicken müssen.

```
#df <- read_csv("C:/Users/christine.dauth/Documents/4_Empirische Wirtschaftsforschung/Einführung in
R/SS_2023/beispieldaten.csv")

# nun ersetzen wird den Pfad mit "pfad" und kommentieren den obigen Schritt aus. Dafür brauchen wir n
och die Funktion paste0():

df <- read_csv(paste0(pfad, "/SS_2023/beispieldaten.csv"))
```

```
## Rows: 273 Columns: 23
## — Column specification —————
## Delimiter: ","
## chr (18): country, g77_and_oecd_countries, income_3groups, income_groups, is...
## dbl (3): iso3166_1_numeric, latitude, longitude
## lgl (2): is--country, un_state
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

3. Der erste Eindruck

Die Daten sind eingelesen und nun wollen wir wissen, wie sie denn aussehen. Folgende Funktionen geben uns einen ersten Eindruck:

```
View(df)
```

Es öffnet sich ein neues Fenster mit dem Datensatz und wir können den gesamten Datensatz überblicken. Alternative Möglichkeiten sind

```
glimpse(df)
```

```
## Rows: 273
## Columns: 23
## $ country      <chr> "abkh", "abw", "afg", "ago", "aia", "akr_a_dhe"...
## $ g77_and_oecd_countries <chr> "others", "others", "g77", "g77", "others", "ot...
## $ income_3groups <chr> NA, "high_income", "low_income", "middle_income...
## $ income_groups <chr> NA, "high_income", "low_income", "lower_middle_...
## $ `is--country` <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE,...
## $ iso3166_1_alpha2 <chr> NA, "AW", "AF", "AO", "AI", NA, "AX", "AL", "AD...
## $ iso3166_1_alpha3 <chr> NA, "ABW", "AFG", "AGO", "AIA", NA, "ALA", "ALB...
## $ iso3166_1_numeric <dbl> NA, 533, 4, 24, 660, NA, 248, 8, 20, NA, 784, 3...
## $ iso3166_2      <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,...
## $ landlocked     <chr> NA, "coastline", "landlocked", "coastline", "co...
## $ latitude       <dbl> NA, 12.50000, 33.00000, -12.50000, 18.21667, NA...
## $ longitude      <dbl> NA, -69.96667, 66.00000, 18.50000, -63.05000, N...
## $ main_religion_2008 <chr> NA, "christian", "muslim", "christian", "christ...
## $ name           <chr> "Abkhazia", "Aruba", "Afghanistan", "Angola", "...
## $ un_sdg_ldc      <chr> NA, "un_not_least_developed", "un_least_develop...
## $ un_sdg_region   <chr> NA, "un_latin_america_and_the_caribbean", "un_c...
## $ un_state        <lgl> FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE, ...
## $ unhcr_region    <chr> NA, "unhcr_americas", "unhcr_asia_pacific", "un...
## $ unicef_region   <chr> NA, NA, "sa", "ssa", NA, NA, NA, "eca", "eca", ...
## $ unicode_region_subtag <chr> NA, "AW", "AF", "AO", "AI", NA, "AX", "AL", "AD...
## $ west_and_rest   <chr> NA, NA, "rest", "rest", NA, NA, NA, "rest", "we...
## $ world_4region   <chr> "europe", "americas", "asia", "africa", "americ...
## $ world_6region   <chr> "europe_central_asia", "america", "south_asia",...
```

```
# oder einfach:
df
```

```
## # A tibble: 273 × 23
##   country g77_and_o...1 incom...2 incom...3 is--c...4 iso31...5 iso31...6 iso31...7 iso31...8
##   <chr>    <chr>      <chr>  <chr>  <lgl>  <chr>  <chr>      <dbl> <chr>
## 1 abkh     others    <NA>  <NA>   TRUE  <NA>  <NA>      NA <NA>
## 2 abw     others  high_i... high_i... TRUE  AW    ABW      533 <NA>
## 3 afg     g77      low_in... low_in... TRUE  AF    AFG      4 <NA>
## 4 ago     g77      middle... lower_... TRUE  AO    AGO      24 <NA>
## 5 aia     others    <NA>  <NA>   TRUE  AI    AIA     660 <NA>
## 6 akr_a_dhe others    <NA>  <NA>   TRUE  <NA>  <NA>      NA <NA>
## 7 ala     others    <NA>  <NA>   TRUE  AX    ALA     248 <NA>
## 8 alb     others  middle... upper_... TRUE  AL    ALB      8 <NA>
## 9 and     others  high_i... high_i... TRUE  AD    AND     20 <NA>
## 10 ant    others    <NA>  <NA>   TRUE  <NA>  <NA>      NA <NA>
## # ... with 263 more rows, 14 more variables: landlocked <chr>, latitude <dbl>,
## # longitude <dbl>, main_religion_2008 <chr>, name <chr>, un_sdg_ldc <chr>,
## # un_sdg_region <chr>, un_state <lgl>, unhcr_region <chr>,
## # unicef_region <chr>, unicode_region_subtag <chr>, west_and_rest <chr>,
## # world_4region <chr>, world_6region <chr>, and abbreviated variable names
## # 1g77_and_oecd_countries, 2income_3groups, 3income_groups, 4`is--country`,
## # 5iso3166_1_alpha2, 6iso3166_1_alpha3, 7iso3166_1_numeric, 8iso3166_2
```

Lernkontrolle:

- Wie viele Variablen enthält der Datensatz?
- Wie viele Beobachtungen enthält der Datensatz?
- Was sind die Merkmalsträger?

4. Wichtige Funktionen beim Aufbereiten und Auswerten von Daten

4.1 Die select-Funktion

Der Datensatz enthält mehr Variablen als uns eigentlich interessieren. Deshalb möchten wir nun einige auswählen und diesen neuen (kleineren) Datensatz unter einem andern Objektnamen abspeichern. Dabei hilft uns die `select()` - Funktion.

```
df_neu <- df %>%  
  select(country, name, income_3groups, income_groups, landlocked, main_religion_2008)
```

Die `select()` -Funktion kann mit weiteren hilfreichen Funktionen verbunden werden. Z.B. `starts_with()` , `contains()` , `ends_with()` . Probieren Sie folgendes aus:

```
dfx <- df %>%  
  select(country, name, contains("iso"))
```

Lernkontrolle:

- Wenden Sie aktiv die Funktionen `starts_with()` , `ends_with()` an, um Daten auszuwählen. Vergessen Sie dabei nicht, die Daten jeweils unter einem Objekt Ihrer Wahl abzuspeichern.

Man kann die `select()` -Funktion auch nutzen, um Variablen im Datensatz umzusortieren. So möchten wir nun, dass die Variable `'name'` am Anfang des Datensatzes steht. Dies erreichen wir mit folgendem Code:

```
df <- df %>%  
  select(name, everything())
```

Wichtig: `everything()` darf nicht fehlen, da ansonsten alle Variablen bis auf `name` aus dem Datensatz gelöscht werden.

Lernkontrolle:

Wir möchten unserem Datensatz `df_neu` nun noch die Info hinzufügen, ob es sich um ein OECD-Land handelt oder nicht und zu welchem Kontinent es gehört:

```
df_neu <- df %>%  
  select(country, name, income_3groups, income_groups, landlocked, main_religion_2008, g77_and_oecd_c  
ountries, world_4region)
```

4.2 Die filter-Funktion

Wir haben den Datensatz nun auf die Variablen beschränkt, die uns wirklich interessieren, d.h. wir haben die Anzahl der Spalten begrenzt. Nun geht es um die Zeilen: wir möchten die Anzahl der Beobachtungen einschränken, da wir uns nur für OECD-Länder interessieren. Diese Länder speichern wir unter `df_oecd` ab.

```
df_oecd <- df_neu %>%  
  filter(g77_and_oecd_countries=="oecd")
```

Lernkontrolle:

Wie viele OECD-Länder sind nun in diesem Datensatz?

4.3 neue Variablen erstellen: `mutate()`, `if_else()`, `case_when()`, `recode()`

Im nächsten Schritt möchten wir dem Datensatz eine neue Variable anhängen, die uns anzeigt, ob in dem Land deutsch gesprochen wird oder nicht. Wir möchten also eine neue Variable selbst generieren, die die Länder Deutschland, Österreich, Schweiz und Italien identifiziert. Dafür benötigen wir folgende Funktionen

```
df_oecd <- df_oecd %>%
  mutate(deutschsp = case_when(
    country == "aut" | country=="che" | country=="deu" | country == "ita" ~ "Ja",
    TRUE ~ "Nein"
  ))
```

Wir haben es uns anders überlegt und wollen der Variable anstelle der Ausprägungen “Ja” und “Nein” doch lieber die Werte 1 und 0 zuweisen. Ganz allgemein gilt: Möchte man eine binäre Variable (nur zwei Ausprägungen) erstellen, hilft die Funktion `if_else()`:

Lernkontrolle:

Erstellen Sie eine Variable `sprache`, die die Ausprägung “deutsch” annimmt bei den vier deutschsprachigen Ländern und die Ausprägung “nicht deutsch” bei allen anderen Ländern.

```
df_oecd <- df_oecd %>%
  mutate(sprache = if_else(
    deutschsp == "Ja", "deutsch", "nicht deutsch")
  )
```

Nun möchten wir die Ausprägungen der Variable `world_4region` ins Deutsche übersetzen. Wir könnten dies mit der Funktion `case_when()` umsetzen, oder aber `recode()` verwenden:

```
df_oecd <- df_oecd %>%
  mutate(region_deu = recode(world_4region,
    "asia" = "Asien",
    "europe" = "Europa",
    "americas" = "Amerika",
    "africa" = "Afrika"))
```

`case_when()` ist die Funktion, mit der am flexibelsten Variablen mit neuen Werten erstellt werden können oder bestehende Variablen umkodiert werden können, da die Booleschen Operatoren verwendet werden können. Wir wollen nun eine Variable `ext` erstellen, die angibt, ob es sich um ein europäisches Land handelt und ob dieses von Wasser umgeben ist oder nicht.

```
df_oecd <- df_oecd %>%
  mutate(ext =case_when(
    world_4region == "europe" & landlocked == "landlocked" ~ "Landlocked Europe",
    world_4region == "europe" & landlocked == "coastline" ~ "Coastline Europe",
    world_4region != "europe" ~ "Not Europe"
  ))

# alternativ:
df_oecd <- df_oecd %>%
  mutate(ext =case_when(
    world_4region == "europe" & landlocked == "landlocked" ~ "Landlocked Europe",
    world_4region == "europe" & landlocked == "coastline" ~ "Coastline Europe",
    TRUE ~ "Not Europe"
  ))
```

Wir möchten nun eine binäre Variable (auch Dummy-Variable genannt) erstellen, die uns anzeigt, ob die dominierende Religion das Christentum ist oder nicht

```
df_oecd <- df_oecd %>%  
  mutate(christ = main_religion_2008=="christian")
```

```
df_oecd %>%  
  slice(1:10)
```

```
## # A tibble: 10 × 14  
##   country name incom...1 incom...2 landl...3 main_...4 g77_a...5 world...6 deuts...7 deuts...8  
##   <chr>   <chr> <chr>   <chr>   <chr>   <chr>   <chr>   <chr>   <chr>   <dbl>  
## 1 aus    Aust... high_i... high_i... coastl... christ... oecd    asia    Nein      0  
## 2 aut    Aust... high_i... high_i... landlo... christ... oecd    europe  Ja       1  
## 3 bel    Belg... high_i... high_i... coastl... christ... oecd    europe  Nein      0  
## 4 can    Cana... high_i... high_i... coastl... christ... oecd    americ... Nein      0  
## 5 che    Swit... high_i... high_i... landlo... christ... oecd    europe  Ja       1  
## 6 cze    Czec... high_i... high_i... landlo... christ... oecd    europe  Nein      0  
## 7 deu    Germ... high_i... high_i... coastl... christ... oecd    europe  Ja       1  
## 8 dnk    Denm... high_i... high_i... coastl... christ... oecd    europe  Nein      0  
## 9 esp    Spain high_i... high_i... coastl... christ... oecd    europe  Nein      0  
## 10 fin    Finl... high_i... high_i... coastl... christ... oecd    europe  Nein      0  
## # ... with 4 more variables: sprache <chr>, region_deu <chr>, ext <chr>,  
## #   christ <lgl>, and abbreviated variable names 1income_3groups,  
## #   2income_groups, 3landlocked, 4main_religion_2008, 5g77_and_oecd_countries,  
## #   6world_4region, 7deutschsp, 8deutschsp_neu
```

4.4 Variablen umbenennen mit rename

Viele Variablenamen sind eher sperrig und lange. Wenn wir wissen, welche Variablen eines Datensatzes für uns wichtig sind und welche Informationen sich genau dahinter verbergen, können wir auch kürzere Variablenamen vergeben, die dann vielleicht aber weniger aufschlussreich über den Inhalt der Werte sind.

```
df_oecd <- df_oecd %>%  
  rename(religion = main_religion_2008,  
         continent = world_4region,  
         oecd_land = g77_and_oecd_countries)
```

4.5 Umgang mit Missing

Oftmals weisen Datensätze viele fehlende Werte auf. Wichtig ist es, zunächst einmal zu verstehen warum diese Werte fehlen. Gibt es systematische Ausfälle oder sind sie eher zufällig? Fehlen viele Werte oder nur wenige? Betreffen fehlende Werte wichtige Variablen oder unwichtige?

Wir gehen zurück zu unserem ursprünglichen Datensatz `df` und betrachten die beiden Variablen `un_state` und `unhcr_region`.

```
df%>%  
  select(un_state, unhcr_region, everything()) %>%  
  View()
```

Es fällt auf, dass die Variable `unhcr_region` häufig dort Missings hat, wo die Variable `un_state` den Wert `FALSE` annimmt. Wenn ein Land nicht Mitglied der Vereinten Nationen ist, dann ist es auch keiner Region des UN-Menschenrechtsrat zugeordnet. Das ist aber scheinbar nicht immer der Fall:

```
df %>%
  count(un_state, unhcr_region)
```

```
## # A tibble: 12 × 3
##   un_state unhcr_region      n
##   <lgl>    <chr>         <int>
## 1 FALSE   unhcr_americas      15
## 2 FALSE   unhcr_asia_pacific    7
## 3 FALSE   unhcr_europe          1
## 4 FALSE   unhcr_middle_east_north_africa 1
## 5 FALSE   <NA>                54
## 6 TRUE    unhcr_americas      35
## 7 TRUE    unhcr_asia_pacific    44
## 8 TRUE    unhcr_east_horn_africa_great_lates 11
## 9 TRUE    unhcr_europe         49
## 10 TRUE   unhcr_middle_east_north_africa 19
## 11 TRUE   unhcr_southern_africa 16
## 12 TRUE   unhcr_west_central_africa 21
```

Wir scheinen aber schon mal nichts falsch zu machen, wenn wir alle Beobachtungen löschen, wo die Variable `unhcr_region` fehlende Werte hat.

```
df_neu <- df %>%
  filter(!is.na(unhcr_region))

df_neu %>%
  count(un_state, unhcr_region)
```

```
## # A tibble: 11 × 3
##   un_state unhcr_region      n
##   <lgl>    <chr>         <int>
## 1 FALSE   unhcr_americas      15
## 2 FALSE   unhcr_asia_pacific    7
## 3 FALSE   unhcr_europe          1
## 4 FALSE   unhcr_middle_east_north_africa 1
## 5 TRUE    unhcr_americas      35
## 6 TRUE    unhcr_asia_pacific    44
## 7 TRUE    unhcr_east_horn_africa_great_lates 11
## 8 TRUE    unhcr_europe         49
## 9 TRUE    unhcr_middle_east_north_africa 19
## 10 TRUE   unhcr_southern_africa 16
## 11 TRUE   unhcr_west_central_africa 21
```

#alternativ:

```
df_neu2 <- df %>%
  drop_na(unhcr_region)
```

Wollen wir untersuchen, ob es Variablen mit vielen Missings gibt, können wir dafür die Funktion `skim()` nutzen. Diese liefert uns ausführliche Informationen zu jeder Variable.

```
skim(df)
```

Data summary

Name	df
------	----

Number of rows	273
Number of columns	23
Column type frequency:	
character	18
logical	2
numeric	3
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
name	0	1.00	3	44	0	273	0
country	0	1.00	3	17	0	273	0
g77_and_oecd_countries	14	0.95	3	6	0	3	0
income_3groups	55	0.80	10	13	0	3	0
income_groups	55	0.80	10	19	0	4	0
iso3166_1_alpha2	25	0.91	2	2	0	248	0
iso3166_1_alpha3	26	0.90	3	3	0	247	0
iso3166_2	272	0.00	5	5	0	1	0
landlocked	18	0.93	9	10	0	2	0
main_religion_2008	57	0.79	6	17	0	3	0
un_sdg_ldc	24	0.91	18	22	0	2	0
un_sdg_region	25	0.91	21	40	0	8	0
unhcr_region	54	0.80	12	34	0	7	0
unicef_region	78	0.71	2	4	0	7	0
unicode_region_subtag	25	0.91	2	2	0	248	0
west_and_rest	78	0.71	4	4	0	2	0
world_4region	2	0.99	4	8	0	4	0
world_6region	13	0.95	7	24	0	6	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
is-country	0	1	1.00	TRU: 273
un_state	0	1	0.71	TRU: 195, FAL: 78

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
iso3166_1_numeric	26	0.90	434.13	253.68	4.0	216.00	434.00	653.00	894.00	
latitude	32	0.88	17.48	25.92	-90.0	1.29	16.75	39.69	78.00	
longitude	32	0.88	14.29	74.25	-176.2	-40.00	17.83	48.00	179.14	

```
skim(df_oecd)
```

Data summary

Name	df_oecd
Number of rows	31
Number of columns	14
Column type frequency:	
character	12
logical	1
numeric	1
Group variables	
None	

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
country	0	1.00	3	3	0	31	0
name	0	1.00	5	15	0	31	0
income_3groups	0	1.00	11	13	0	2	0
income_groups	0	1.00	11	19	0	2	0
landlocked	0	1.00	9	10	0	2	0
religion	2	0.94	6	17	0	3	0
oecd_land	0	1.00	4	4	0	1	0
continent	0	1.00	4	8	0	3	0
deutschsp	0	1.00	2	4	0	2	0
sprache	0	1.00	7	13	0	2	0
region_deu	0	1.00	5	7	0	3	0
ext	0	1.00	10	17	0	3	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
christ	2	0.94	0.93	TRU: 27, FAL: 2

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
deutschsp_neu	0	1	0.13	0.34	0	0	0	0	1	

5 Deskriptive Statistiken erstellen

5.1 Erste Statistiken für quantitative Variablen mit `summarize()`

Wenn wir mit Datensätzen arbeiten, wollen wir nicht nur Variablen anpassen und verändern (Datenaufbereitung), wir wollen Daten auch auswerten. Erste wichtige Statistiken liefert uns die `summarize()`-Funktion. Wir wechseln nun zu einem anderen (altbekannten) Datensatz, der numerische/ quantitative Variablen enthält.

```
# summary statistics
View(flights)

distance_summary <- flights %>%
  summarize(
    mean = mean(distance, na.rm = TRUE),
    sd = sd(distance, na.rm = TRUE),
    min = min(distance, na.rm = TRUE),
    q1 = quantile(distance, 0.25, na.rm = TRUE),
    median = quantile(distance, 0.5, na.rm = TRUE),
    q3 = quantile(distance, 0.75, na.rm = TRUE),
    max = max(distance, na.rm = TRUE),
    missing = sum(is.na(distance))
  )

distance_summary
```

```
## # A tibble: 1 × 8
##   mean    sd  min   q1 median   q3   max missing
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>   <int>
## 1 1040.  733.   17   502   872  1389  4983     0
```

Wichtig ist, dass wir uns gezielt um fehlende Werte (NA) kümmern müssen, wenn wir `summarize` verwenden.

Lernkontrolle

Welche Statistiken werden für welche Variable berechnet? Wie können wir diese interpretieren?

5.2 Statistiken differenzieren mit `group_by()`

Wir haben die wichtigsten Statistiken zur Flugdistanz ausgewertet. Informativer wäre es jetzt noch auszuwerten, ob die durchschnittliche Distanz nach Abflugsflughafen variiert. Dies können wir mit `group_by` auswerten:

```
distance_by_origin <- flights %>%
  group_by(origin) %>%
  summarize(
    mean = mean(distance, na.rm = TRUE),
    sd = sd(distance, na.rm = TRUE),
    min = min(distance, na.rm = TRUE),
    q1 = quantile(distance, 0.25, na.rm = TRUE),
    median = quantile(distance, 0.5, na.rm = TRUE),
    q3 = quantile(distance, 0.75, na.rm = TRUE),
    max = max(distance, na.rm = TRUE),
    missing = sum(is.na(distance))
  )

distance_by_origin
```

```
## # A tibble: 3 × 9
##   origin mean    sd  min   q1 median   q3  max missing
##   <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>   <int>
## 1 EWR    1057.  730.   17  529   872  1400  4963     0
## 2 JFK    1266.  896.   94  427  1069  2248  4983     0
## 3 LGA     780.  372.   96  502   762  1035  1620     0
```

Die Funktion `group_by()` gehört zum Paket `dplyr`, das den Datensatz/ data frame gruppiert. Die Funktion `group_by()` allein liefert keinen Output. Nach `group_by()` sollte die Funktion `summarize()` mit einer geeigneten Aktion ausgeführt werden.

5.3. Häufigkeiten qualitativer Variablen auszählen mit `count()`

Als nächstes wollen wir wissen, welcher Zielflughafen (`dest`) am häufigsten angeflogen wird. Dazu wollen wir auswerten, wie häufig jeweils ein Zielflughafen im Datensatz vorkommt. Wir zählen also aus, wie viele Flüge es pro Zielflughafen gibt.

```
freq_dest <- flights %>%
  group_by(dest) %>%
  summarize(num_flights = n())
freq_dest
```

```
## # A tibble: 105 × 2
##   dest num_flights
##   <chr>      <int>
## 1 ABQ         254
## 2 ACK         265
## 3 ALB         439
## 4 ANC          8
## 5 ATL       17215
## 6 AUS       2439
## 7 AVL         275
## 8 BDL         443
## 9 BGR         375
## 10 BHM        297
## # ... with 95 more rows
```

```
# eine deutlich kürzere Alternative um Werte auszuzählen:
freq_dest2 <- flights %>%
  count(dest)
freq_dest
```

```
## # A tibble: 105 × 2
##   dest  num_flights
##   <chr>      <int>
## 1 ABQ         254
## 2 ACK         265
## 3 ALB         439
## 4 ANC          8
## 5 ATL       17215
## 6 AUS       2439
## 7 AVL         275
## 8 BDL         443
## 9 BGR         375
## 10 BHM        297
## # ... with 95 more rows
```

5.4 Statistiken sortieren mit `arrange()` und `arrange(desc())`

Die Werte sind alphabetisch nach dem Wert des Zielflughafens sortiert. So können wir aber nicht erkennen, welcher Flughafen am häufigsten angefliegen wurde. Damit das jetzt noch übersichtlicher wird, müssen wir die Zahlen sortieren:

```
# vom kleinsten zum größten Wert der Variable num_flights sortieren:
freq_dest %>%
  arrange(num_flights)
```

```
## # A tibble: 105 × 2
##   dest  num_flights
##   <chr>      <int>
## 1 LEX          1
## 2 LGA          1
## 3 ANC          8
## 4 SBN         10
## 5 HDN         15
## 6 MTJ         15
## 7 EYW         17
## 8 PSP         19
## 9 JAC         25
## 10 BZN         36
## # ... with 95 more rows
```

```
# vom größten zum kleinsten Wert der Variable num_flights sortieren:
freq_dest %>%
  arrange(desc(num_flights))
```

```
## # A tibble: 105 × 2
##   dest  num_flights
##   <chr>      <int>
## 1 ORD      17283
## 2 ATL      17215
## 3 LAX      16174
## 4 BOS      15508
## 5 MCO      14082
## 6 CLT      14064
## 7 SFO      13331
## 8 FLL      12055
## 9 MIA      11728
## 10 DCA       9705
## # ... with 95 more rows
```

5.5 Statistiken mehrere Variablen gleichzeitig berechnen

Wir können nun schon die wichtigsten Statistiken für eine Variable berechnen. Das ist jedoch etwas wenig; meistens möchten wir einen schnellen Überblick über die Werte sämtlicher Variablen erhalten. Wir lernen nun verschiedene Funktionen kennen, mit denen das möglich ist. Wir fangen an mit `skim()`.

```
skim(flights)
```

Data summary

Name	flights
Number of rows	336776
Number of columns	19
Column type frequency:	
character	4
numeric	14
POSIXct	1
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
carrier	0	1.00	2	2	0	16	0
tailnum	2512	0.99	5	6	0	4043	0
origin	0	1.00	3	3	0	3	0
dest	0	1.00	3	3	0	105	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1.00	2013.00	0.00	2013	2013	2013	2013	2013	
month	0	1.00	6.55	3.41	1	4	7	10	12	
day	0	1.00	15.71	8.77	1	8	16	23	31	
dep_time	8255	0.98	1349.11	488.28	1	907	1401	1744	2400	
sched_dep_time	0	1.00	1344.25	467.34	106	906	1359	1729	2359	
dep_delay	8255	0.98	12.64	40.21	-43	-5	-2	11	1301	
arr_time	8713	0.97	1502.05	533.26	1	1104	1535	1940	2400	
sched_arr_time	0	1.00	1536.38	497.46	1	1124	1556	1945	2359	
arr_delay	9430	0.97	6.90	44.63	-86	-17	-5	14	1272	
flight	0	1.00	1971.92	1632.47	1	553	1496	3465	8500	
air_time	9430	0.97	150.69	93.69	20	82	129	192	695	
distance	0	1.00	1039.91	733.23	17	502	872	1389	4983	
hour	0	1.00	13.18	4.66	1	9	13	17	23	
minute	0	1.00	26.23	19.30	0	8	29	44	59	

Variable type: POSIXct

skim_variable	n_missing	complete_rate	min	max	median	n_unique
time_hour	0	1	2013-01-01 05:00:00	2013-12-31 23:00:00	2013-07-03 10:00:00	6936

Lernkontrolle

Interpretieren Sie den kompletten Output.

`skim()` gibt uns einen guten Überblick, aber leider ist der Output nicht besonders ansehlich. Generell empfiehlt sich folgende Art der Darstellung:

1. Quantitative Variablen: wichtige Maße in einer Tabelle zusammenfassen
2. Qualitative Variablen: Häufigkeitstabellen (ein- oder zweidimensional (Kreuztabellen))

5.5.1 Quantitative Variablen

Eine Alternative zu `skim()` kommt aus dem Paket `stargazer`. Dieses Paket berechnet Statistiken *für numerische Variablen* und erstellt gleichzeitig einen ansehlichen Output, der in ein Latex- oder Word-Dokument übernommen werden kann.

```
setwd(graf) # Pfad angeben, damit Output dort gespeichert werden kann
stargazer(as.data.frame(flights), summary.stat = NULL, median = TRUE, type = "text", title="Descriptive statistics", digits=1, out="table1.html")
```



```
##
## Descriptive statistics
## =====
## Statistic      N      Mean  St. Dev.  Min  Median  Max
## -----
## year          336,776 2,013.0   0.0     2,013 2,013  2,013
## month         336,776   6.5     3.4       1     7    12
## day           336,776  15.7     8.8       1    16    31
## dep_time      328,521 1,349.1  488.3     1   1,401  2,400
## sched_dep_time 336,776 1,344.3  467.3    106   1,359  2,359
## dep_delay     328,521  12.6     40.2    -43    -2   1,301
## arr_time      328,063 1,502.1  533.3     1   1,535  2,400
## sched_arr_time 336,776 1,536.4  497.5     1   1,556  2,359
## arr_delay     327,346   6.9     44.6   -86    -5   1,272
## flight        336,776 1,971.9 1,632.5     1   1,496  8,500
## air_time      327,346  150.7    93.7     20    129   695
## distance      336,776 1,039.9  733.2     17    872  4,983
## hour          336,776  13.2     4.7       1     13    23
## minute        336,776  26.2    19.3       0     29    59
## -----
```

Der gewünschte Output (welche Statistiken, welcher Style, Punkt oder Komma als Trennzeichen,...) kann noch angepasst werden. Dafür einfach in die R-Hilfe schauen!

In dem Arbeitsverzeichnis liegt nun eine html-Datei mit dem Namen table1.html. Um die Tabelle nun in ein Word-Dokument einzufügen, öffnen wir diese Datei im Browser, kopieren die Tabelle und fügen Sie dann einfach in das Word-Dokument ein. Nun können wir die Tabelle noch bearbeiten, z.B. die Variablennamen noch informativer labeln.

5.5.2 Qualitative Variablen

Nachdem wir nun wissen, wie wir effizient Statistiken für **quantitative** Variablen exportieren können, wollen wir uns um **qualitative** Variablen kümmern. Wir haben bereits `summarize()` und `count()` kennengelernt und damit absolute Häufigkeiten ausgezählt. Wir können mit den bereits kennengelernten Funktionen auch relative und kumulierte Häufigkeitstabellen gut darstellen. Inhaltlich geht es uns in folgendem Beispiel darum, darzustellen, wie viele Flüge aus NYC im Jahr 2013 jeweils einen bestimmten Zielflughafen hatten. Das hatten wir bereits, aber neu ist nun, dass wir die absoluten Werte aufsteigend sortieren und die relativen und kumulierten Häufigkeiten hinzufügen.

Neben der Funktion `n()`, die absolute Häufigkeiten berechnet, benötigen wir nun noch die Funktion `sum()`, die Werte komplett aufsummiert und die Funktion `cumsum()`, die kumuliert aufsummiert.

```
flights %>%
  group_by(dest) %>%
  summarize(num_flights = n()) %>%
  arrange(num_flights) %>%
  mutate(
    # neue Spalte berechnen, die die Summe der Flüge nach Ankunftsort enthält
    total_n = sum(num_flights),
    # Anteil pro Ankunftsflughafen
    prop = num_flights/total_n,
    # nun noch die kumulativen Häufigkeiten ausweisen
    cum_prop = cumsum(prop)
  )
```

```
## # A tibble: 105 × 5
##   dest num_flights total_n      prop cum_prop
##   <chr>      <int>   <int>    <dbl>   <dbl>
## 1 LEX          1 336776 0.00000297 0.00000297
## 2 LGA          1 336776 0.00000297 0.00000594
## 3 ANC          8 336776 0.0000238 0.0000297
## 4 SBN         10 336776 0.0000297 0.0000594
## 5 HDN         15 336776 0.0000445 0.000104
## 6 MTJ         15 336776 0.0000445 0.000148
## 7 EYW         17 336776 0.0000505 0.000199
## 8 PSP         19 336776 0.0000564 0.000255
## 9 JAC         25 336776 0.0000742 0.000330
## 10 BZN        36 336776 0.000107 0.000436
## # ... with 95 more rows
```

Eine alternative Funktion aus dem Paket `janitor` ist `tabyl()`, die uns neben den absoluten Häufigkeiten auch prozentuale Häufigkeiten ausweist.

```
flights %>%
  tabyl(dest) %>%
  adorn_pct_formatting() %>%
  top_n(10)
```

```
## Warning: Using one column matrices in `filter()` was deprecated in dplyr 1.1.0.
## i Please use one dimensional logical vectors instead.
## i The deprecated feature was likely used in the dplyr package.
## Please report the issue at <[8];https://github.com/tidyverse/dplyr/issues[8];https://github.com/tidyverse/dplyr/issues[8];[8]>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Selecting by percent
```

```
##   dest      n percent
##   ATL 17215    5.1%
##   BOS 15508    4.6%
##   CLT 14064    4.2%
##   DCA  9705    2.9%
##   FLL 12055    3.6%
##   LAX 16174    4.8%
##   MCO 14082    4.2%
##   MIA 11728    3.5%
##   ORD 17283    5.1%
##   SFO 13331    4.0%
```

Beachten Sie: mit `top_n(10)` geben wir an, dass nur die ersten 10 Zeilen des Outputs angezeigt werden sollen.

Neben der eindimensionalen Häufigkeitstabelle können wir auch Kreuztabellen erstellen. Hier zuerst eine Variante, bei der zwar die Häufigkeiten zweier Variablen ausgezählt werden, aber die Darstellung nicht besonders übersichtlich ist.

```
# nicht so schön:
hfmt1 <- flights %>%
  group_by(dest, origin) %>%
  summarize(num_flights = n()) %>%
  arrange(dest, origin)
```

```
## `summarise()` has grouped output by 'dest'. You can override using the
## `.groups` argument.
```

```
hfkt1
```

```
## # A tibble: 224 × 3
## # Groups:   dest [105]
##   dest origin num_flights
##   <chr> <chr>      <int>
## 1 ABQ   JFK         254
## 2 ACK   JFK         265
## 3 ALB   EWR         439
## 4 ANC   EWR           8
## 5 ATL   EWR        5022
## 6 ATL   JFK        1930
## 7 ATL   LGA       10263
## 8 AUS   EWR         968
## 9 AUS   JFK       1471
## 10 AVL  EWR         265
## # ... with 214 more rows
```

```
hfkt1 <- hfkt1 %>%
  group_by(dest) %>%
  mutate(
    # neue Spalte berechnen, die die Summe der Flüge nach Ankunftsort enthält
    total_n = sum(num_flights),
    # Anteil pro Ankunftsflughafen
    prop = num_flights/total_n,
    # nun noch die kumulativen Häufigkeiten ausweisen
    cum_prop = cumsum(prop)
  )

print(hfkt1)
```

```
## # A tibble: 224 × 6
## # Groups:   dest [105]
##   dest origin num_flights total_n prop cum_prop
##   <chr> <chr>      <int>   <int> <dbl>   <dbl>
## 1 ABQ   JFK         254     254 1      1
## 2 ACK   JFK         265     265 1      1
## 3 ALB   EWR         439     439 1      1
## 4 ANC   EWR           8       8 1      1
## 5 ATL   EWR        5022    17215 0.292  0.292
## 6 ATL   JFK        1930    17215 0.112  0.404
## 7 ATL   LGA       10263    17215 0.596  1
## 8 AUS   EWR         968     2439 0.397  0.397
## 9 AUS   JFK       1471     2439 0.603  1
## 10 AVL  EWR         265       275 0.964  0.964
## # ... with 214 more rows
```

5.5.3 Kreuztabelle in Excel exportieren

Eine alternative Möglichkeit steht uns mit `tabyl` zur Verfügung. In diesem Zusammenhang wollen wir uns auch ansehen, wie wir diese Tabelle in Excel exportieren können.

```
# Variante: nur absolute Werte
tabelle0 <- flights %>%
  tabyl(dest,origin, show_na = TRUE, show_missing_levels = TRUE) %>%
  adorn_totals(c("row", "col")) %>%
  adorn_title("combined")

tabelle0 %>%
  top_n(15)
```

```
## Selecting by Total
```

```
## dest/origin    EWR    JFK    LGA  Total
##          ATL  5022   1930  10263  17215
##          BOS  5327   5898   4283  15508
##          CLT  5026   2870   6168  14064
##          DCA  1719   3270   4716   9705
##          DFW  3148    732   4858   8738
##          DTW  3178   1166   5040   9384
##          FLL  3793   4254   4008  12055
##          LAX  4912  11262     0  16174
##          MCO  4941   5464   3677  14082
##          MIA  2633   3314   5781  11728
##          ORD  6100   2326   8857  17283
##          RDU  1482   3100   3581   8163
##          SFO  5127   8204     0  13331
##          TPA  2334   2987   2145   7466
##          Total 120835 111279 104662 336776
```

```
# Variante: absolute und prozentuale Werte
tabelle1 <- flights %>%
  tabyl(dest,origin, show_na = TRUE, show_missing_levels = TRUE) %>%
  adorn_totals(c("row", "col")) %>%
  adorn_title("combined") %>%
  adorn_percentages("row") %>%
  adorn_pct_formatting(digits = 2) %>%
  adorn_ns()

tabelle1 %>%
  top_n(15)
```

```
## Selecting by Total
```

##	dest/origin		EWB		JFK		LGA		Total
##	ATL	29.17%	(5022)	11.21%	(1930)	59.62%	(10263)	100.00%	(17215)
##	BOS	34.35%	(5327)	38.03%	(5898)	27.62%	(4283)	100.00%	(15508)
##	CLT	35.74%	(5026)	20.41%	(2870)	43.86%	(6168)	100.00%	(14064)
##	DCA	17.71%	(1719)	33.69%	(3270)	48.59%	(4716)	100.00%	(9705)
##	DFW	36.03%	(3148)	8.38%	(732)	55.60%	(4858)	100.00%	(8738)
##	DTW	33.87%	(3178)	12.43%	(1166)	53.71%	(5040)	100.00%	(9384)
##	FLL	31.46%	(3793)	35.29%	(4254)	33.25%	(4008)	100.00%	(12055)
##	LAX	30.37%	(4912)	69.63%	(11262)	0.00%	(0)	100.00%	(16174)
##	MCO	35.09%	(4941)	38.80%	(5464)	26.11%	(3677)	100.00%	(14082)
##	MIA	22.45%	(2633)	28.26%	(3314)	49.29%	(5781)	100.00%	(11728)
##	ORD	35.29%	(6100)	13.46%	(2326)	51.25%	(8857)	100.00%	(17283)
##	RDU	18.16%	(1482)	37.98%	(3100)	43.87%	(3581)	100.00%	(8163)
##	SFO	38.46%	(5127)	61.54%	(8204)	0.00%	(0)	100.00%	(13331)
##	TPA	31.26%	(2334)	40.01%	(2987)	28.73%	(2145)	100.00%	(7466)
##	Total	35.88%	(120835)	33.04%	(111279)	31.08%	(104662)	100.00%	(336776)

Nun wollen wir die Tabelle nach Excel exportieren.

Zunächst legen wir uns ein `workbook` für ein Excel-Dokument an.

```
wb <- createWorkbook()
```

Dann fügen wir die Tabellenblätter hinzu, die wir benötigen. In diesem Fall erstellen wir zwei Tabellenblätter

```
addWorksheet(wb, sheet = "abs Anzahl Flüge nach Ziel", gridLine = FALSE)
addWorksheet(wb, sheet = "abs/rel Anzahl Flüge nach Ziel", gridLine = FALSE)
```

Dann schreiben wir den Output in dieses Tabellenblatt:

```
writeData(wb, sheet = "abs Anzahl Flüge nach Ziel", x = tabelle0, startCol= 1, startRow = 1, rowNames
=TRUE, borders = "surrounding")
writeData(wb, sheet = "abs/rel Anzahl Flüge nach Ziel", x = tabelle1, startCol= 1, startRow = 1, rowN
ames=TRUE, borders = "surrounding")
```

Schließlich navigieren wir zu unserem Arbeitsverzeichnis und speichern dort die Excel-Datei ab.

```
setwd(graf)
saveWorkbook(wb, "auswertung_beispiel.xlsx", overwrite = TRUE)
```

Bereits beim Exportieren in Excel kann man in RStudio schon ziemlich gut festlegen, wie die Tabelle später aussehen soll. Ein Beispiel für die Möglichkeiten zeigt dieses Video (<https://www.youtube.com/watch?v=MdhSxQmm4ZU>).

6. Daten visualisieren

Die Visualisierung von Daten und Zahlen wird zunehmend wichtiger und populärer. Ein Vorteil von R ist, dass ansehnliche Grafiken sehr leicht erstellt werden können. Im Folgenden wollen wir uns kurz die wichtigsten Arten von Grafiken anschauen (Ismay und Kim 2023 - <https://moderndive.com/index.html> (<https://moderndive.com/index.html>)).

6.1 Scatterplots

Wir bleiben bei unserem Datensatz zu Flügen in den USA und wollen wissen: Welchen Zusammenhang gibt es zwischen der Verspätung beim Abflug (x-Achse) und der Verspätung bei der Ankunft (y-Achse)?

Um dies zu tun, betrachten wir beispielhaft nur die Flüge der Alaska Airlines flights, die in NYC2013 abgehoben sind, um die Anzahl der Beobachtungen zu verringern.

```
View(flights)
glimpse(flights)
```

```
## Rows: 336,776
## Columns: 19
## $ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2...
## $ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
## $ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
## $ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ...
## $ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ...
## $ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -1...
## $ arr_time  <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849,...
## $ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851,...
## $ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -1...
## $ carrier   <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "...
## $ flight    <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4...
## $ tailnum   <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394...
## $ origin    <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA", "...
## $ dest      <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD", "...
## $ air_time  <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1...
## $ distance  <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ...
## $ hour      <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6...
## $ minute    <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0...
## $ time_hour <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-01 0...
```

```
alaska_flights <- flights %>%
  filter(carrier == "AS")
```

Lernkontrolle

Vergleichen Sie die Datensätze `alaska_flights` und `flights` : Wie unterscheiden Sie sich?

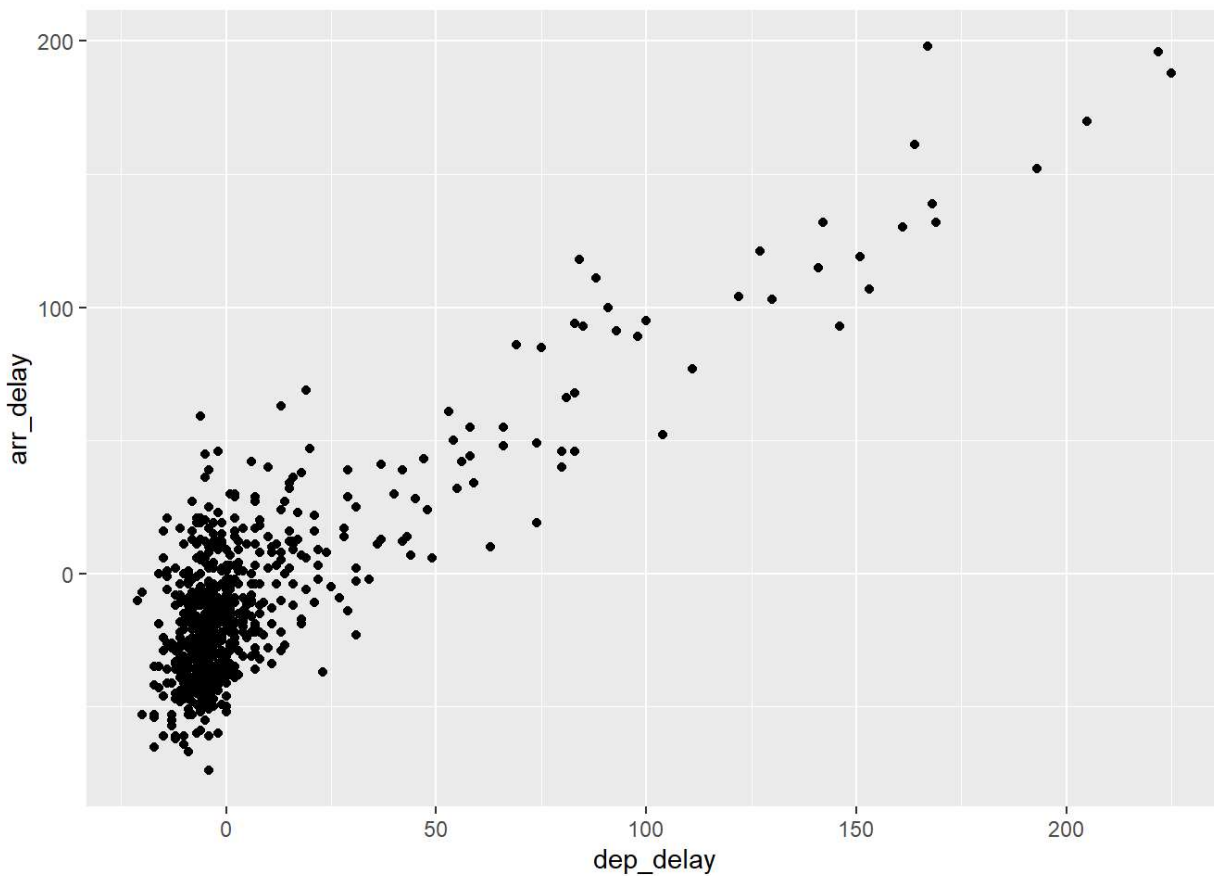
```
View(flights)
View(alaska_flights)
```

Wir erstellen alle Grafiken im Folgenden mit `ggplot2` mit der bekannten Struktur. Folgende Dinge müssen wir innerhalb von `ggplot` spezifizieren:

- Datensatz/ data frame
- aesthetic mapping (welche Variablen werden auf x- und y-Achse abgebildet?)
- Ebene (welche Art von Grafik soll dargestellt werden), indem wir `+` verwenden

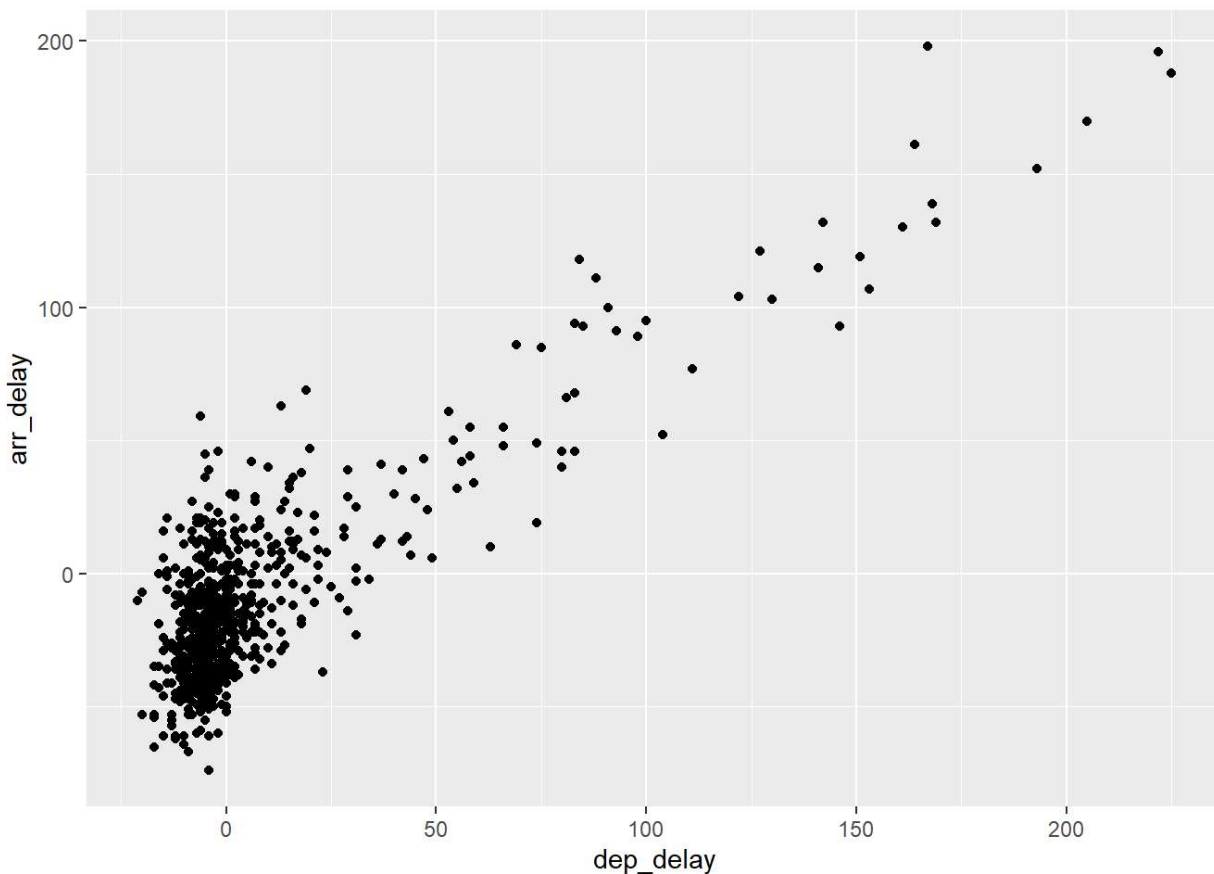
```
ggplot(data = alaska_flights, mapping = aes(x = dep_delay, y = arr_delay)) +  
  geom_point()
```

```
## Warning: Removed 5 rows containing missing values (`geom_point()`).
```



```
# oder kürzer:  
ggplot(alaska_flights, aes(dep_delay, arr_delay)) + geom_point()
```

```
## Warning: Removed 5 rows containing missing values (`geom_point()`).
```



Lernkontrolle

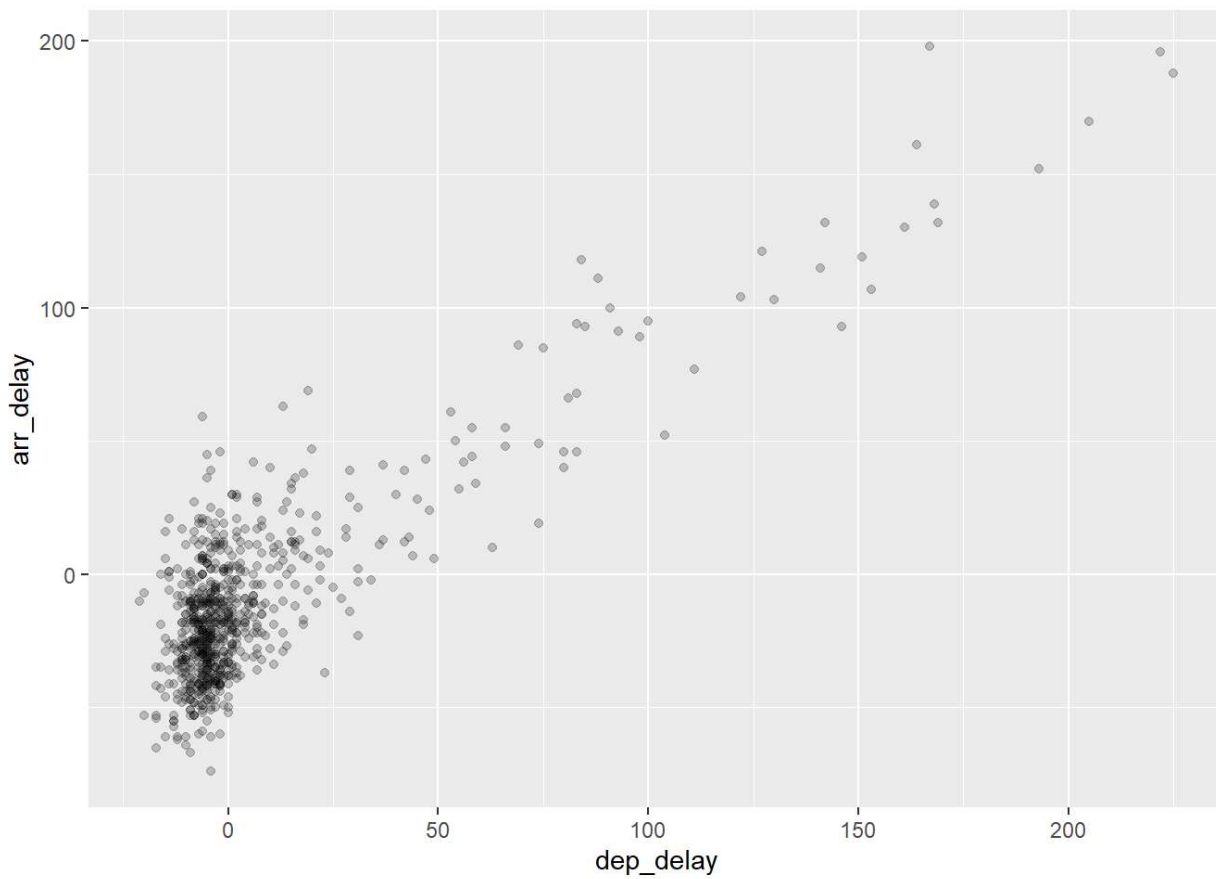
- Wie sieht die Grafik aus, wenn wir keine Ebene spezifizieren?
- Aus welchen praktischen Gründen scheint es zwischen `dep_delay` und `arr_delay` einen positive Zusammenhang zu geben?
- Welche Variablen im `weather` sind wahrscheinlich negativ mit `dep_delay` korreliert? Hinweis: nutzen Sie `View(weather)`.
- Warum häufen sich die Punkte bei (0,0)? Wie lässt sich das inhaltlich interpretieren?
- Erstellen Sie einen neuen Scatterplot mit anderen Variablen aus dem `alaska-flights` Datensatz.

Wenn viele Beobachtungspunkte übereinander liegen, spricht man auch von “Overplotting”. Um einen Eindruck davon zu gewinnen, wie viel Beobachtungsmasse sich an einer Stelle befindet, gibt es zwei Möglichkeiten.

1. Möglichkeit: man verändert die Transparenz der Beobachtungspunkte:

```
ggplot(data = alaska_flights, mapping = aes(x = dep_delay, y = arr_delay)) +  
  geom_point(alpha = 0.2)
```

```
## Warning: Removed 5 rows containing missing values (`geom_point()`).
```

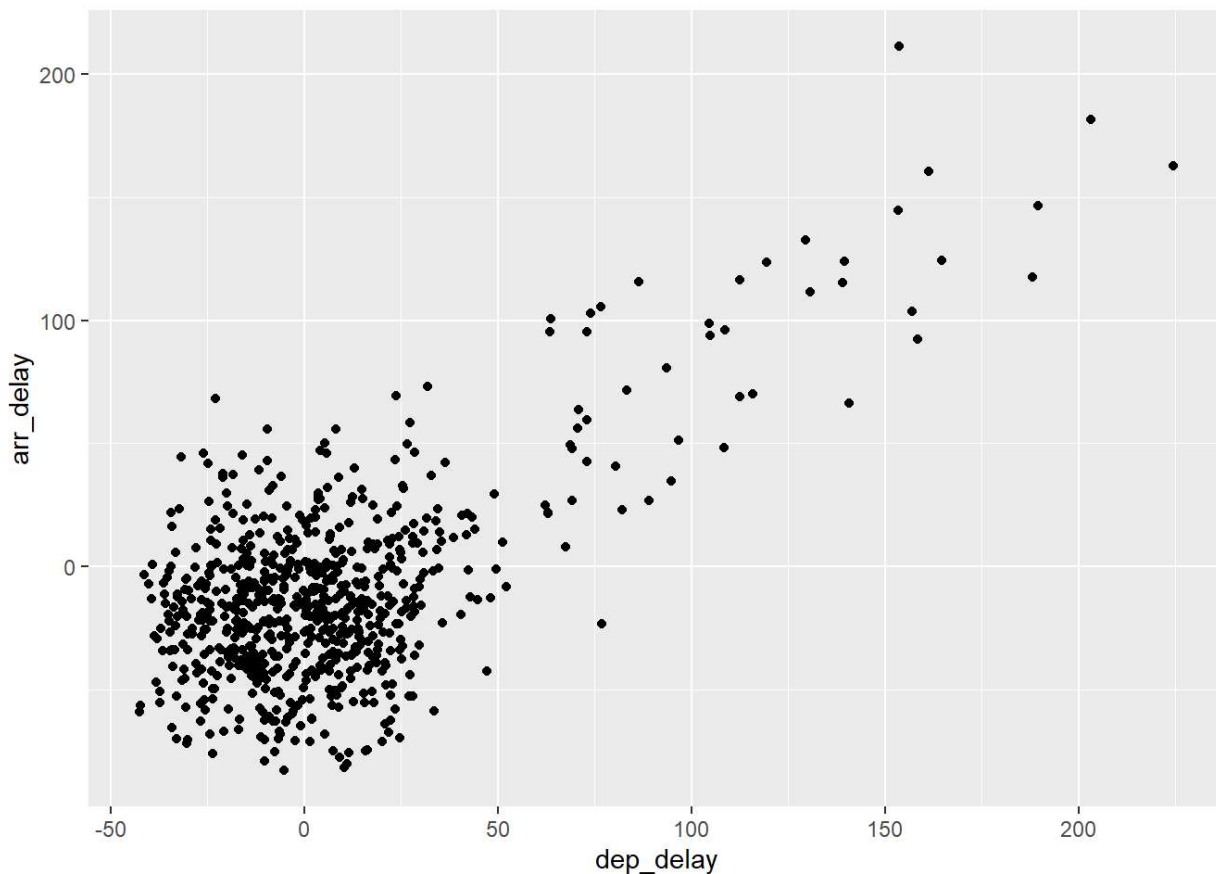
```
#Change transparency:
```

```
#alpha between 0 and 1: 0: 100% transparent, 1: 100% opaque (1 is default setting)
```

2. Möglichkeit: man verwackelt die Punkte etwas ("Jitter")

```
ggplot(data = alaska_flights, mapping = aes(x = dep_delay, y = arr_delay)) +  
  geom_jitter(width = 30, height = 30)
```

```
## Warning: Removed 5 rows containing missing values (`geom_point()`).
```



Da "Jitter" die tatsächlichen Werte verändert, ist diese Möglichkeit nur für den persönlichen Eindruck geeignet, in einer Arbeit sollten nur die wahren Werte dargestellt werden.

Lernkontrolle

- Welche zusätzlichen Informationen erhalten Sie durch `alpha`, die ein normales Streudiagramm nicht liefern kann?
- Geben Sie nach der Betrachtung des transparenten Plots einen ungefähren Bereich der Ankunftsverzögerungen an, die am häufigsten auftreten.
- Wie hat sich dieser Bereich im Vergleich zur Betrachtung desselben Diagramms ohne `alpha = 0,2` verändert?

6.2 Liniendiagramme

Wir wollen nun die Zeitreihe stündlicher Temperaturen in einem Liniendiagramm darstellen. Zur Vereinfachung konzentrieren wir uns auf die Temperaturen am Newark Flughafen in den ersten 15 Tagen im Januar.

```
View(weather)
glimpse(weather)
```

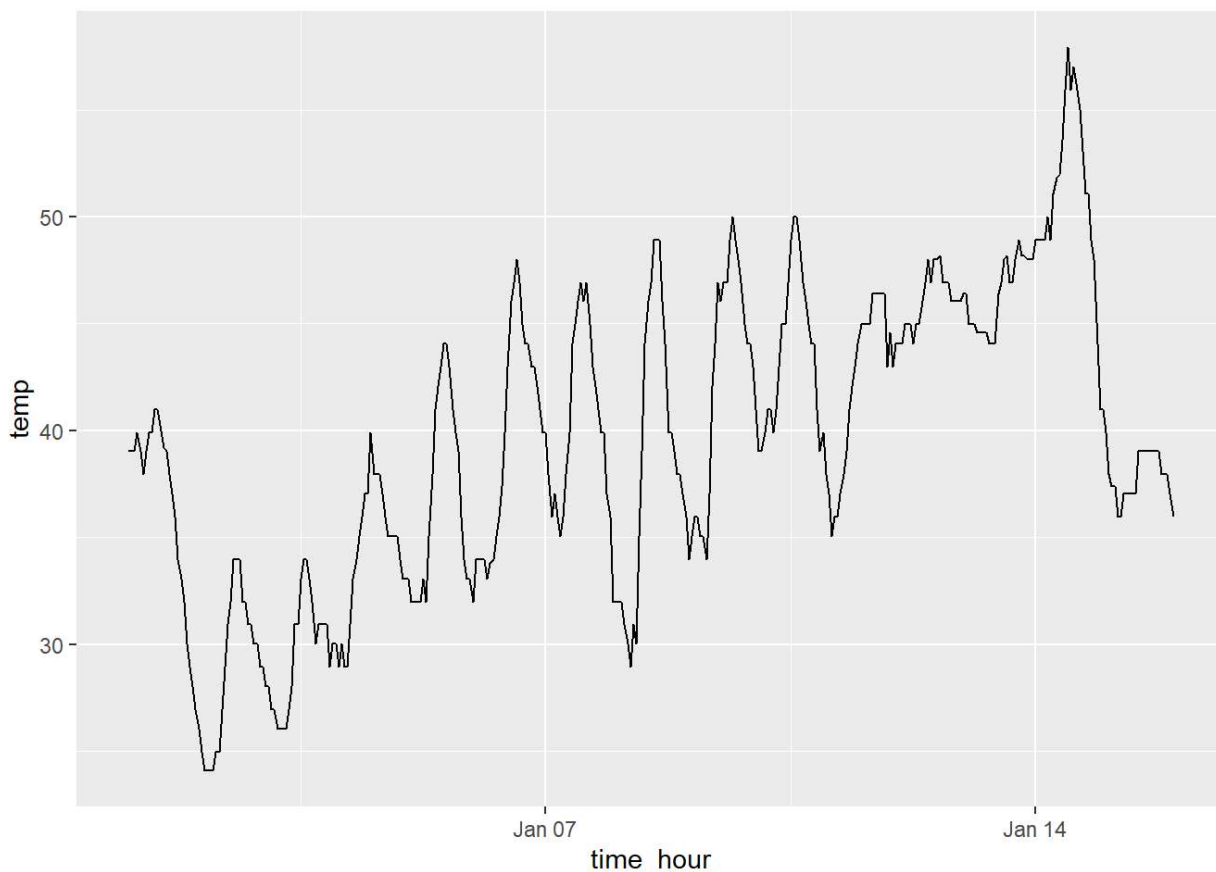
```
## Rows: 26,115
## Columns: 15
## $ origin      <chr> "EWR", "EWR", "EWR", "EWR", "EWR", "EWR", "EWR", "EWR", "EW...
## $ year        <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013,...
## $ month       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
## $ day         <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
## $ hour        <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, ...
## $ temp        <dbl> 39.02, 39.02, 39.02, 39.92, 39.02, 37.94, 39.02, 39.92, 39....
## $ dewp        <dbl> 26.06, 26.96, 28.04, 28.04, 28.04, 28.04, 28.04, 28.04, 28....
## $ humid       <dbl> 59.37, 61.63, 64.43, 62.21, 64.43, 67.21, 64.43, 62.21, 62....
## $ wind_dir    <dbl> 270, 250, 240, 250, 260, 240, 240, 250, 260, 260, 260, 330,...
## $ wind_speed  <dbl> 10.35702, 8.05546, 11.50780, 12.65858, 12.65858, 11.50780, ...
## $ wind_gust   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 20....
## $ precip      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...
## $ pressure    <dbl> 1012.0, 1012.3, 1012.5, 1012.2, 1011.9, 1012.4, 1012.2, 101...
## $ visib       <dbl> 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10,...
## $ time_hour   <dtm> 2013-01-01 01:00:00, 2013-01-01 02:00:00, 2013-01-01 03:00...
```

?weather

```
## starte den http Server für die Hilfe fertig
```

```
early_january_weather <- weather %>%
  filter(origin == "EWR" & month == 1 & day <= 15)

ggplot(data = early_january_weather, mapping = aes(x = time_hour, y = temp)) +
  geom_line()
```



Lernkontrolle:

- Warum sollten Liniendiagramme vermieden werden, wenn es keine klare Rangfolge der Werte der x-Achse gibt?
- Warum werden Liniendiagramme häufig verwendet, wenn es sich bei der Variable auf der x-Achse um Zeit handelt?
- Stellen Sie die Zeitreihe einer anderen Variable als `temp` dar.

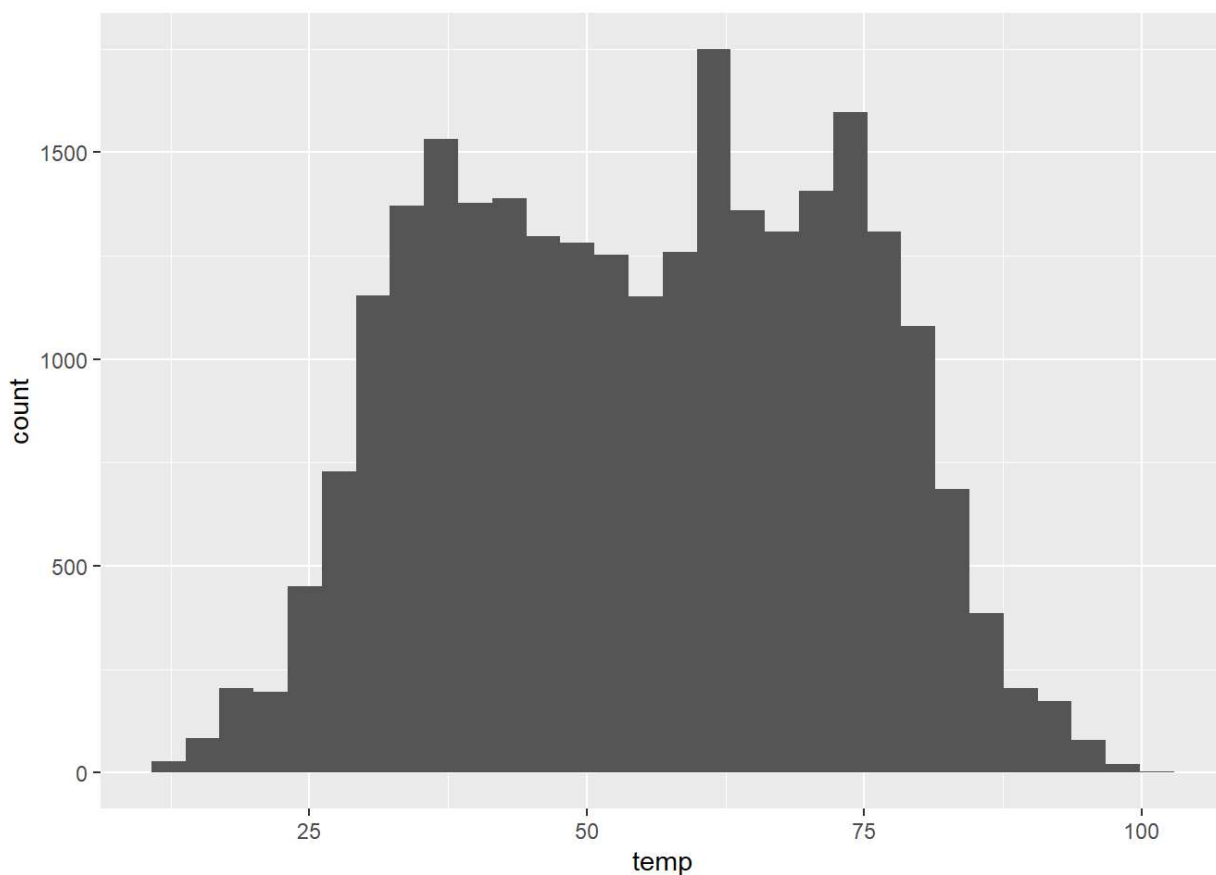
6.3 Histogramme

Histogramme zeigen uns die Häufigkeitsverteilung von stetigen Variablen. Wir können nun ein Histogramm der Temperaturen erstellen, um zu sehen, welche Temperaturen wie oft gemessen wurden.

```
ggplot(data = weather, mapping = aes(x = temp)) +  
  geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

```
## Warning: Removed 1 rows containing non-finite values (`stat_bin()`).
```

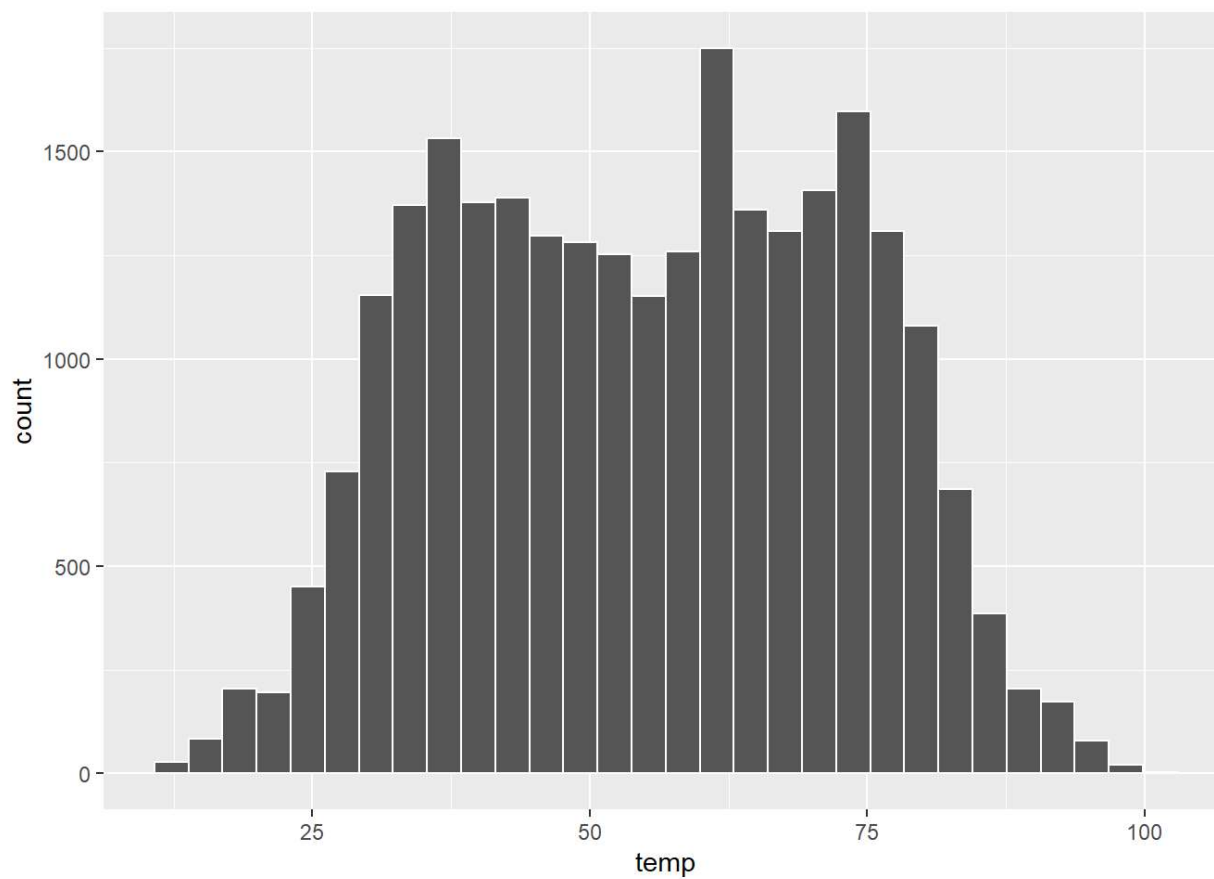


Die Grafik sieht noch nicht so hübsch aus. Was können wir verbessern? Wir fügen vertikale Grenzen ein, um die Verteilung visuell zu unterteilen und färben die Säulen ein.

```
ggplot(data = weather, mapping = aes(x = temp)) +  
  geom_histogram(color = "white")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

```
## Warning: Removed 1 rows containing non-finite values (`stat_bin()`).
```

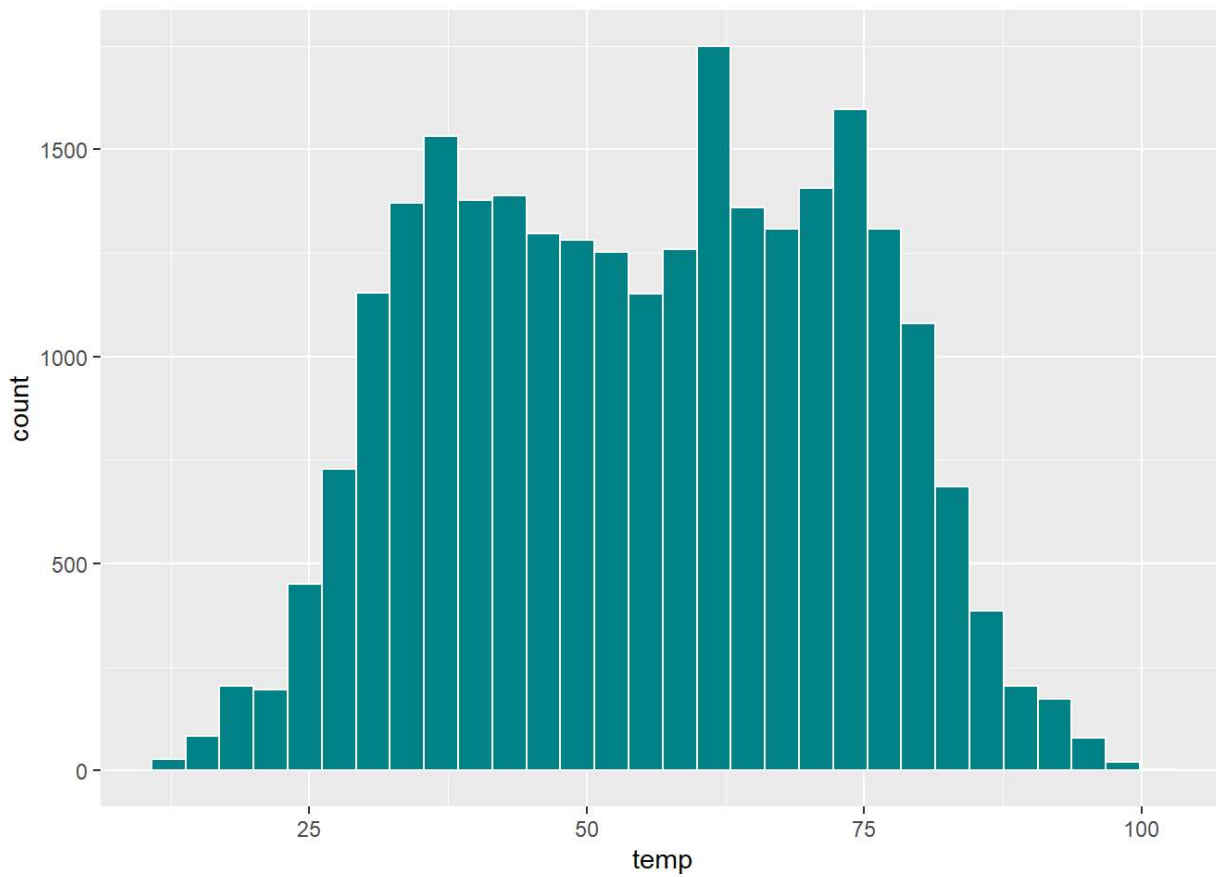


```
#colors()
```

```
ggplot(data = weather, mapping = aes(x = temp)) +  
  geom_histogram(color = "white", fill = "turquoise4")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

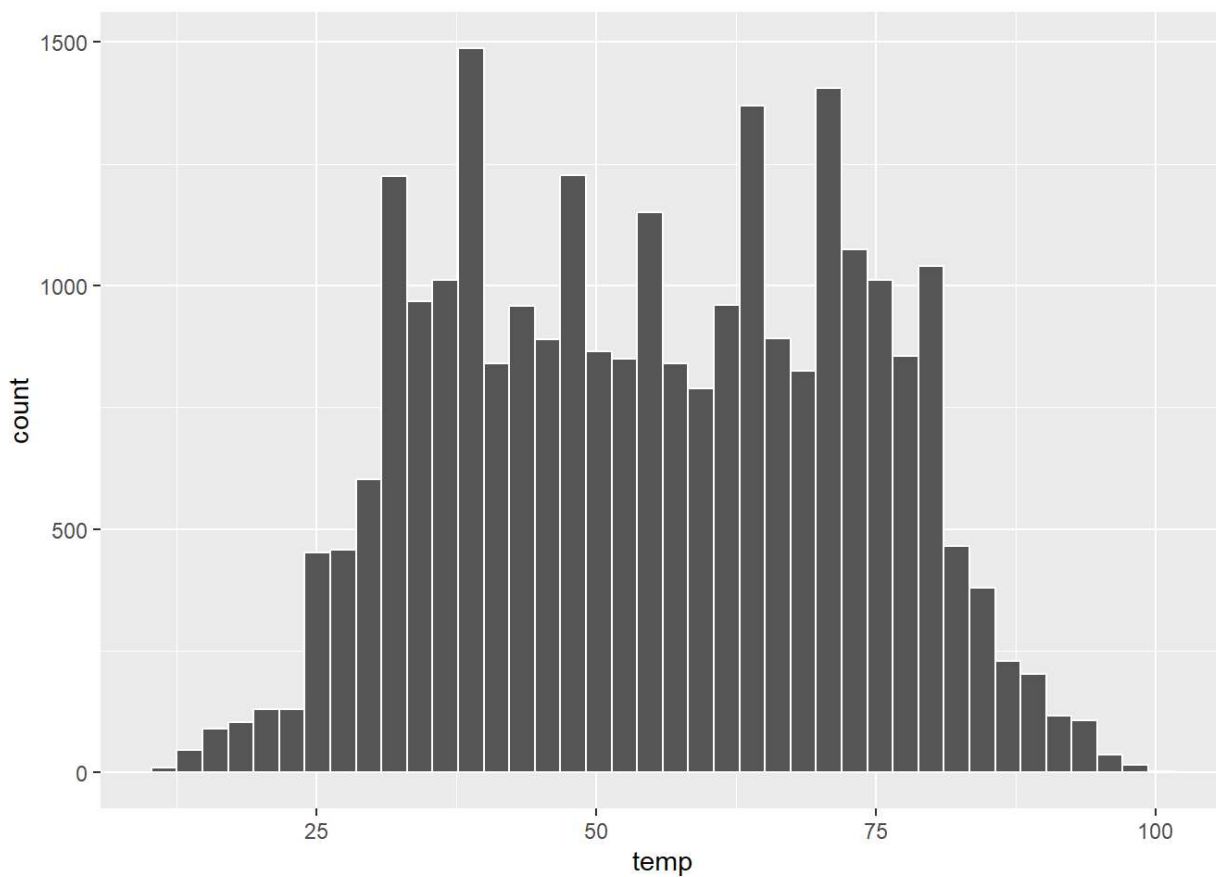
```
## Warning: Removed 1 rows containing non-finite values (`stat_bin()`).
```



Wir können auch die Anzahl der Säulen ("bins") angeben, wenn uns die Darstellung zu ungenau ist.

```
ggplot(data = weather, mapping = aes(x = temp)) +  
  geom_histogram(bins = 40, color = "white")
```

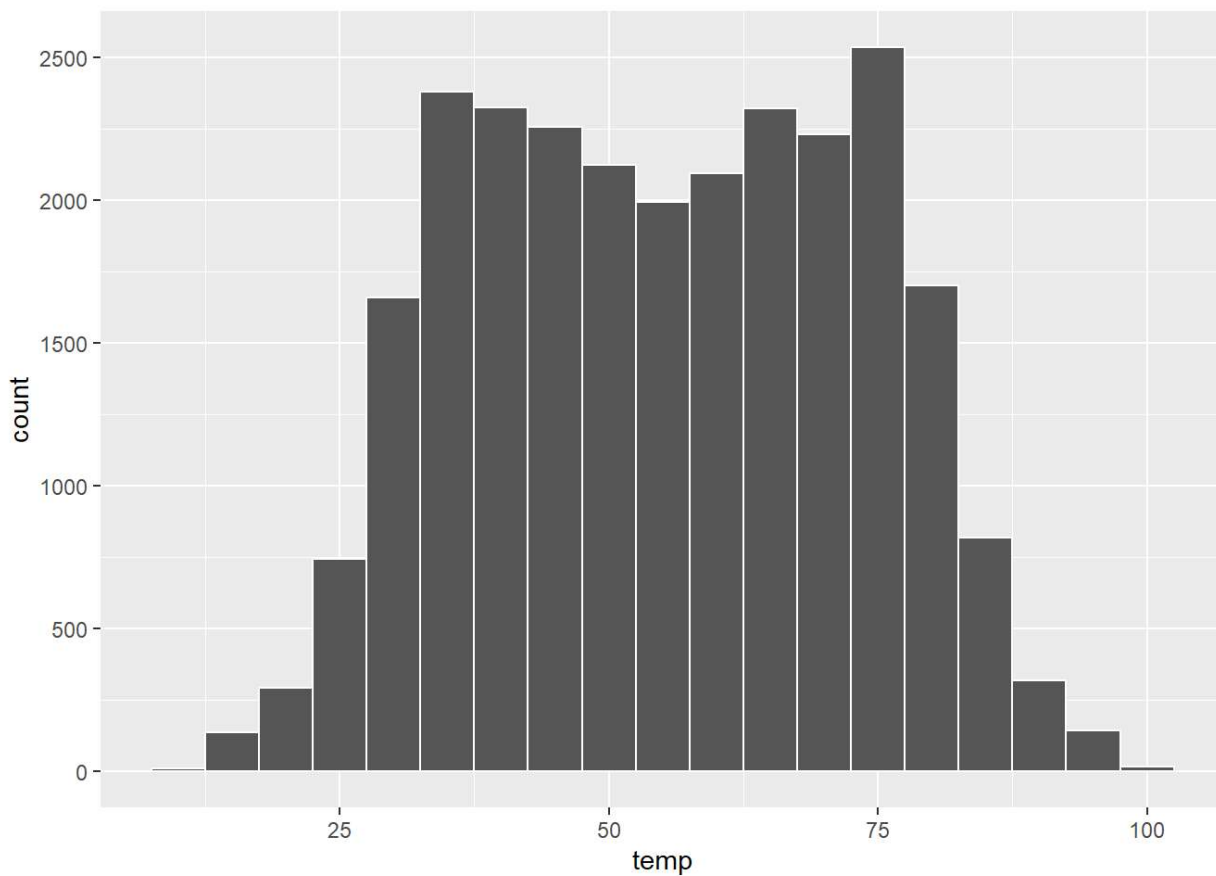
```
## Warning: Removed 1 rows containing non-finite values (`stat_bin()`).
```



Alternativ können wir auch die Breite der Bins angeben. Wir wollen z.B. Bins mit einer Breite von 5 Grad Fahrenheit.

```
ggplot(data = weather, mapping = aes(x = temp)) +  
  geom_histogram(binwidth = 5, color = "white")
```

```
## Warning: Removed 1 rows containing non-finite values (`stat_bin()`).
```



Lernkontrolle

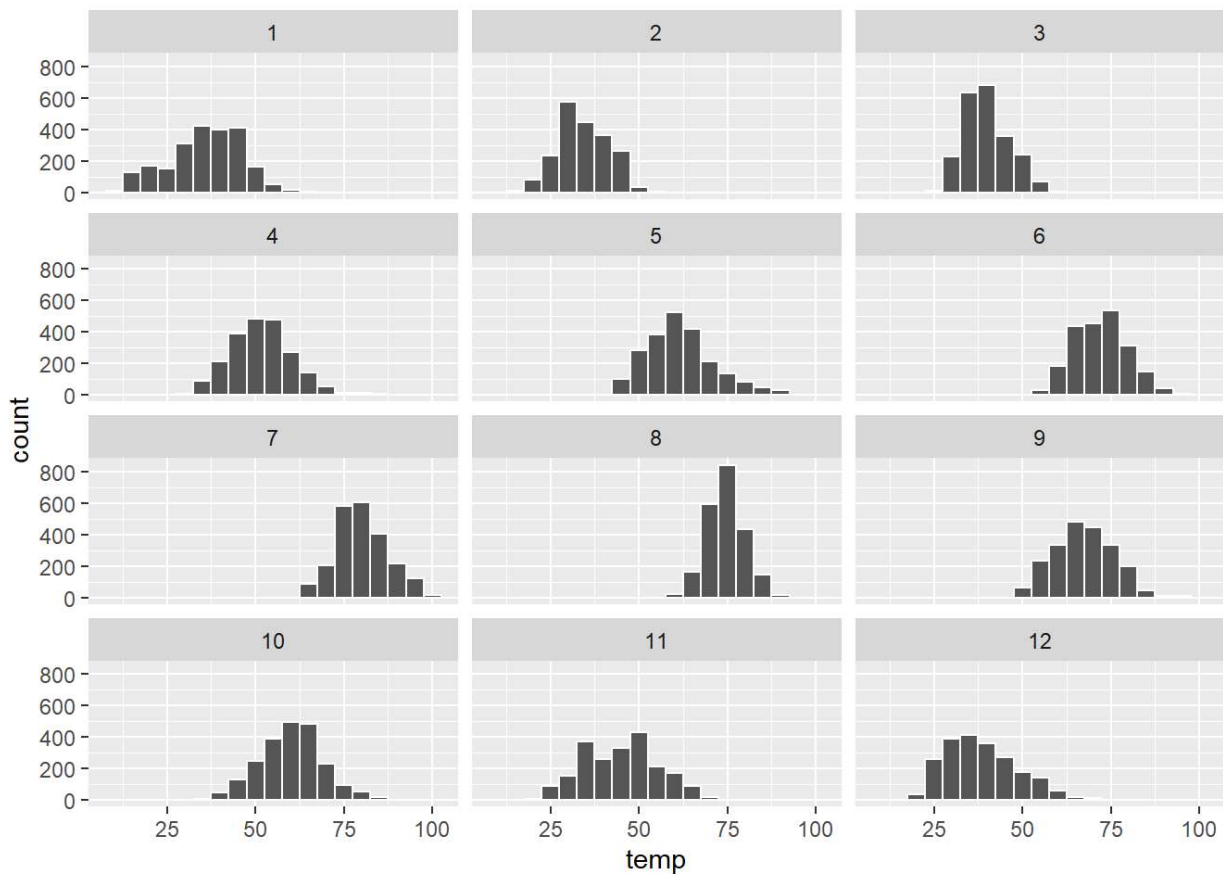
- Was sagt die Änderung der Anzahl der Bins von 30 auf 40 über die Verteilung der Temperaturen aus?
- Würden Sie die Temperaturverteilung als symmetrisch oder in die eine oder andere Richtung verzerrt einstufen?
- Was ist Ihrer Meinung nach der "mittlere" Wert in dieser Verteilung? Warum haben Sie diese Wahl getroffen?

6.4 Facets

Bisher ist die Grafik wenig aussagekräftig, da sich die Werte auf ein ganzes Jahr beziehen. Wir können die Aussagekraft der Darstellung steigern, wenn wir darstellen, wie sich die Temperaturen jeweils nach Kalendermonaten verteilen.

```
ggplot(data=weather, mapping = aes(x =temp)) +  
  geom_histogram(binwidth =5, color = "white") +  
  facet_wrap(~ month, nrow = 4)
```

```
## Warning: Removed 1 rows containing non-finite values (`stat_bin()`).
```



Lernkontrolle

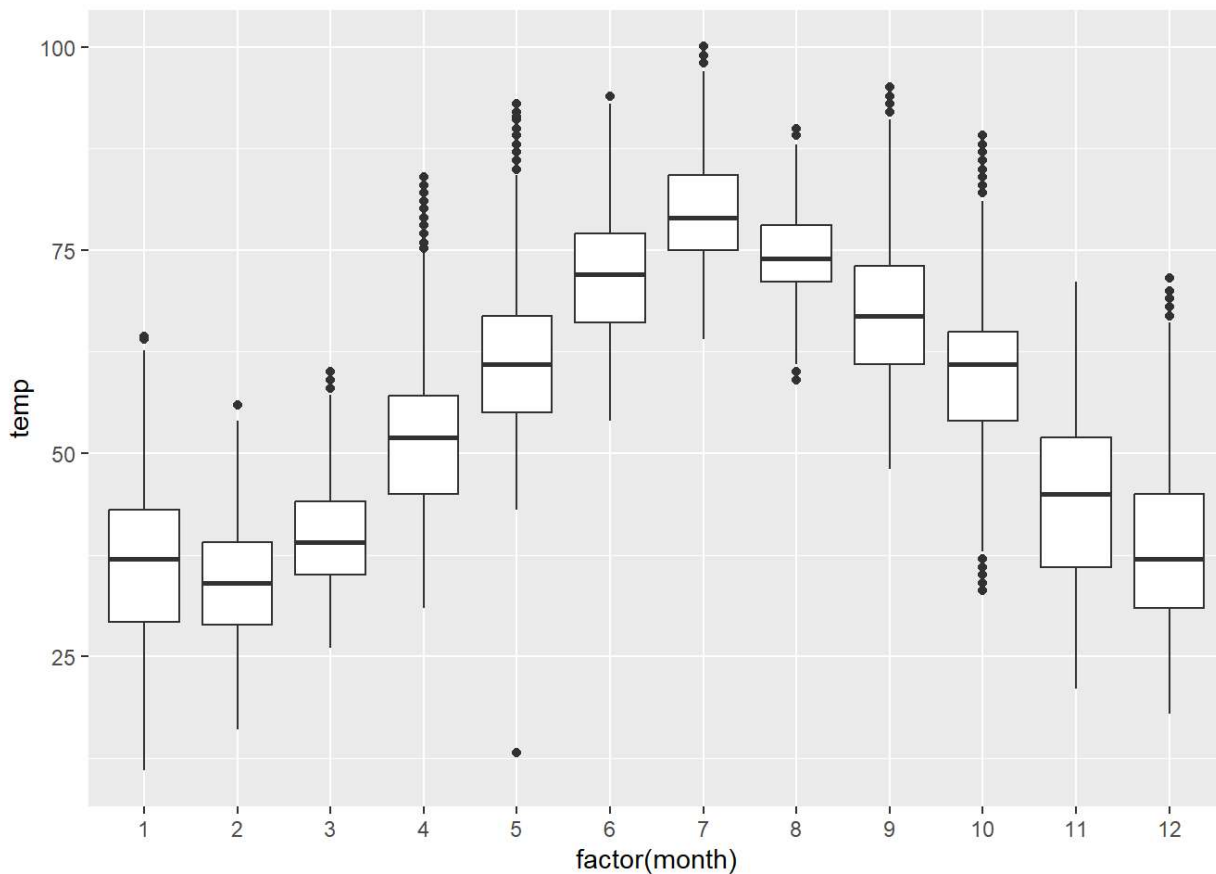
- Was fällt Ihnen an diesem Diagramm auf (Jahreszeiten/Verteilungen)?
- Was bedeuten die Zahlen 1-12 in der Grafik? Was bedeuten 25, 50, 75, 100?
- Für welche Arten von Datensätzen eignet sich diese Art der Darstellungen nicht gut, um Beziehungen zwischen Variablen zu vergleichen?
- Weist die Variable "Temperatur" eine große Variabilität auf? Weshalb?

6.5 Boxplots

Boxplots sind eine übersichtliche alternative Form der Darstellung von Verteilungen von stetigen Variablen. Wir erstellen nun nebeneinander Boxplots für die stündlichen Temperaturen für jedes Kalendermonat.

```
ggplot(data = weather, mapping = aes(x = factor(month), y = temp)) +  
  geom_boxplot()
```

```
## Warning: Removed 1 rows containing non-finite values (`stat_boxplot()`).
```

Lernkontrolle

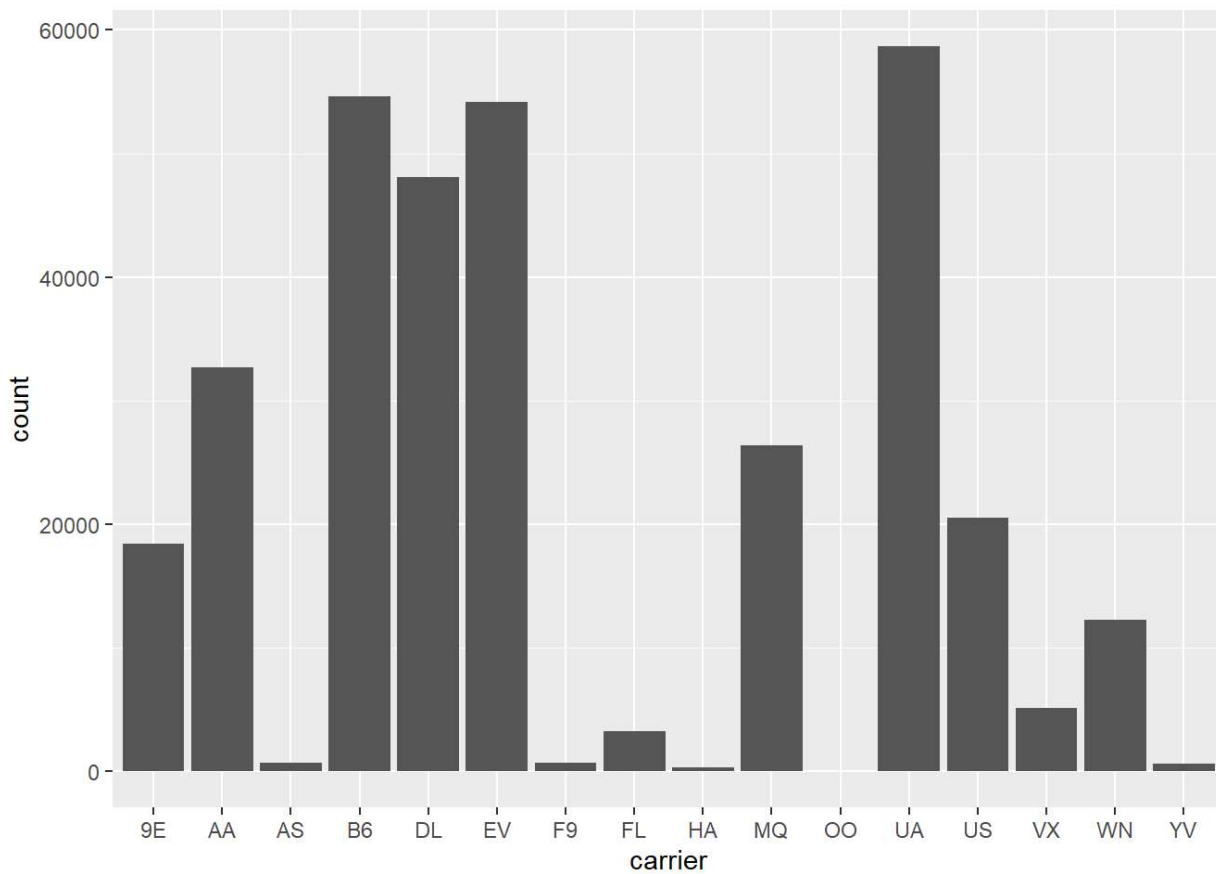
- Was bedeutet der Punkt am unteren Rand der Grafik Mai? Erklären Sie, was passiert sein könnte.
- Welche Monate weisen die größten Temperaturschwankungen auf? Welche Gründe können Sie dafür nennen?
- Wir haben uns die Verteilung der numerischen Variable Temperatur, getrennt nach der numerischen Variable Monat, angesehen, die wir mit der Funktion `factor()` umgewandelt haben, um einen Boxplot nebeneinander zu erstellen. Warum wäre ein Boxplot der Verteilung der Temperatur, geteilt durch die numerische Variable Druck, die ebenfalls mit der Funktion `factor()` in eine kategoriale Variable umgewandelt wurde, nicht informativ?
- Boxplots bieten eine einfache Möglichkeit, Ausreißer zu identifizieren. Warum können Ausreißer leichter identifiziert werden, wenn ein Boxplot anstelle von Facet-Histogrammen betrachtet wird?

6.6 Säulendiagramme

Bisher haben wir nur quantitative Variablen grafisch dargestellt. Wenn wir qualitative Variablen auszählen möchten, dann empfiehlt sich die Darstellung mittels Säulendiagrammen.

Wir möchten nun die Verteilung der Variable `carrier` aus dem `flights`-Datensatz grafisch darstellen.

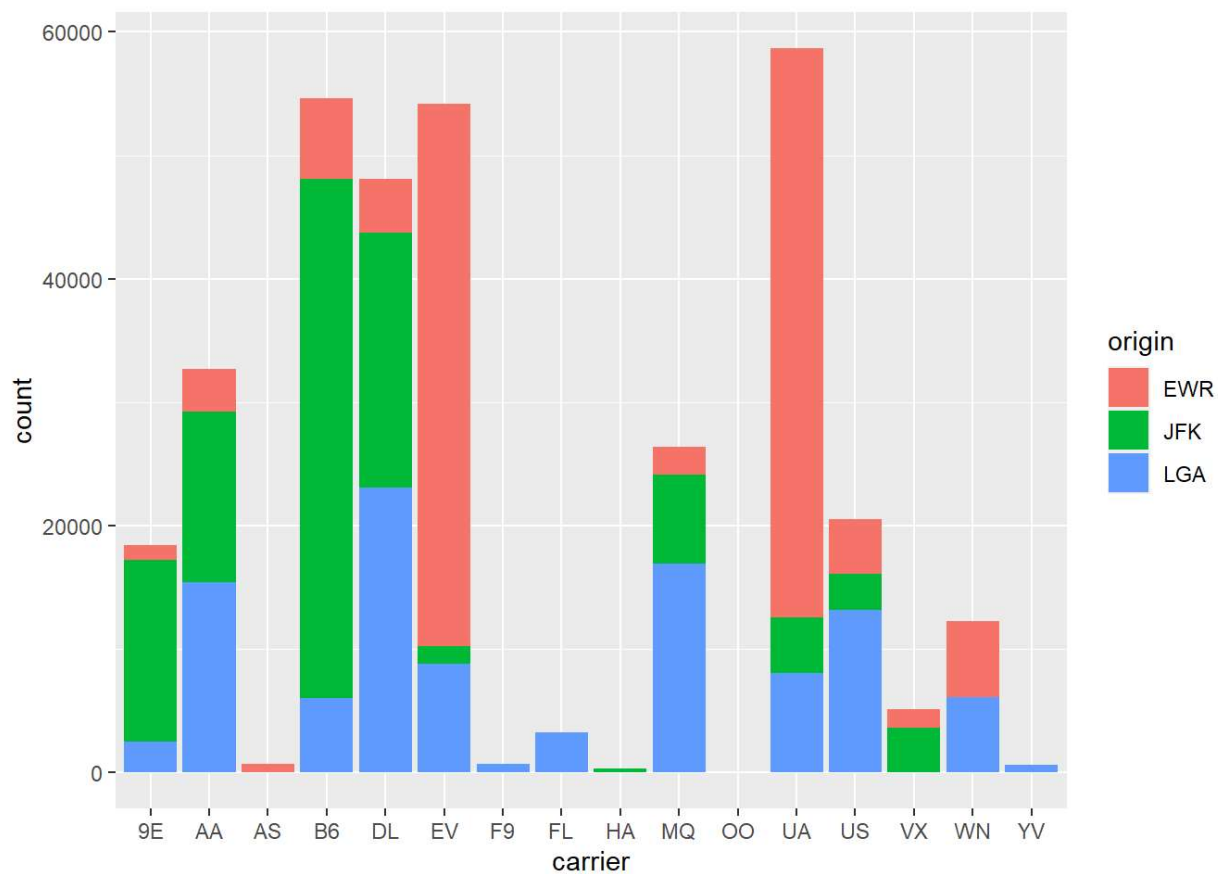
```
ggplot(data = flights, mapping = aes(x = carrier)) +
  geom_bar()
```



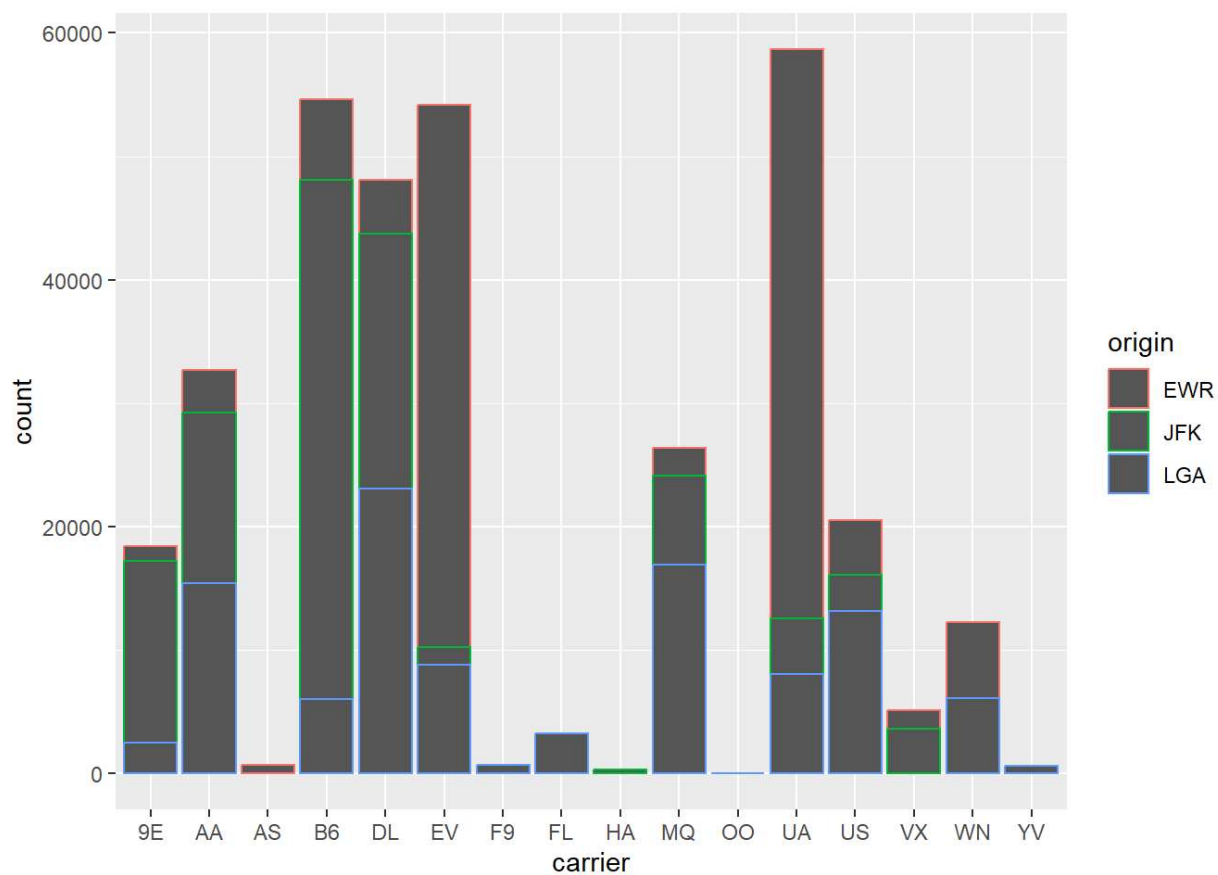
Lernkontrolle Was zeigt uns diese Grafik an?

Nun wollen wir noch anzeigen, wo die Flüge jeweils gestartet sind. Diese Info steckt in der Variable `origin`. Wir erstellen ein gestapeltes Balkendiagramm. Damit können wir leicht sehen, welche Fluggesellschaft welchen Flughafen häufig nutzt.

```
ggplot(data = flights, mapping = aes(x = carrier, fill = origin)) +  
  geom_bar()
```



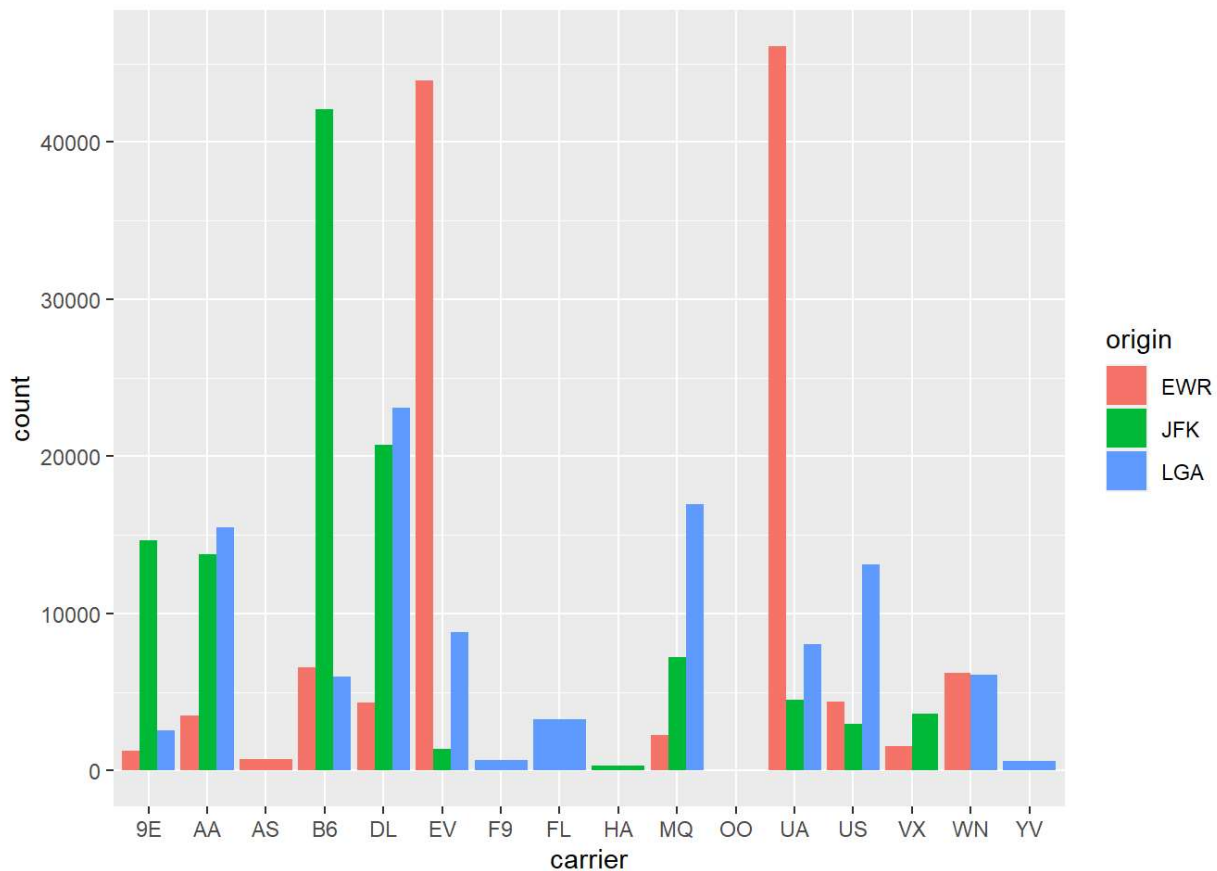
```
ggplot(data = flights, mapping = aes(x = carrier, color = origin)) +
  geom_bar()
```



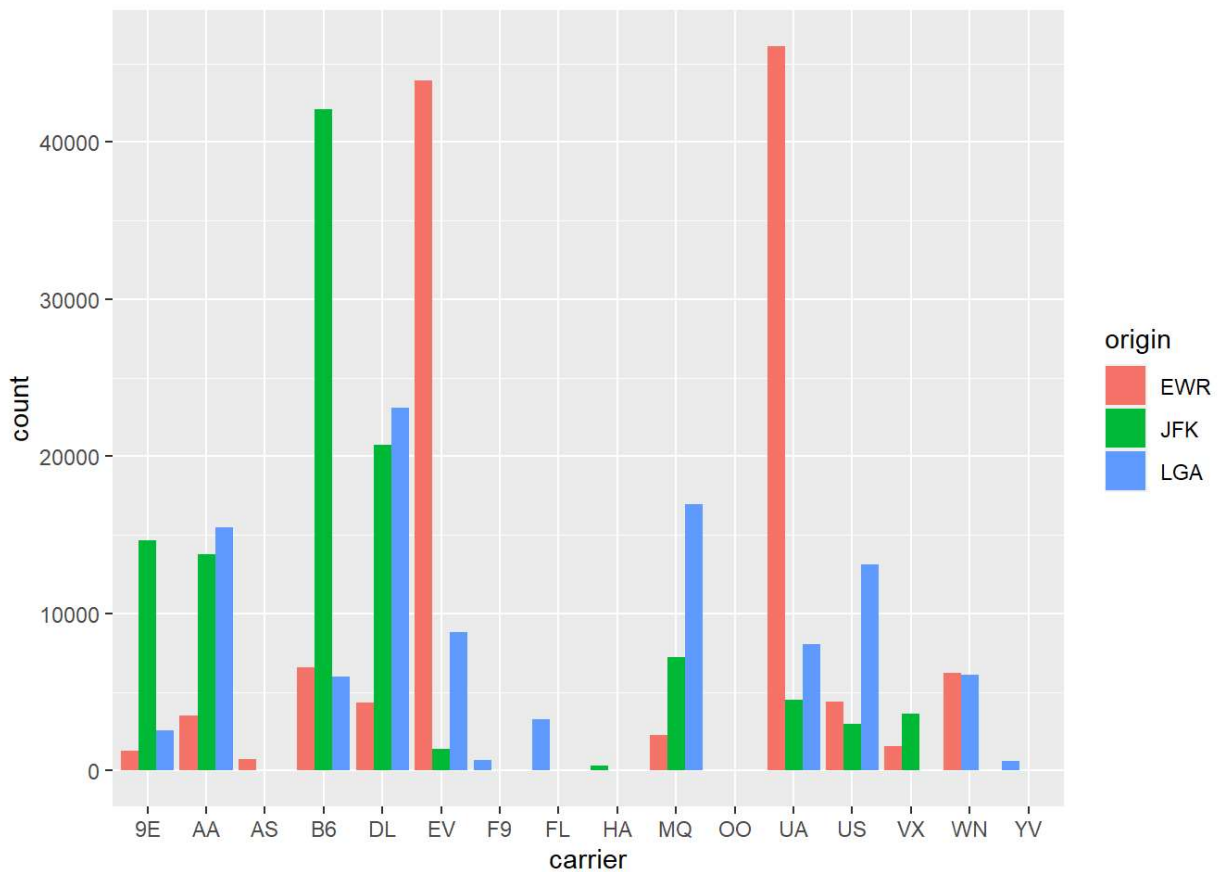
ATTENTION:
color: color of the outline of the bars
fill: color used to fill the bars --> is part of the aesthetic mapping!

Alternativ können wir die Balkendiagramme auch nebeneinander anzeigen lassen: Aus dieser Grafik geht sehr gut hervor, welche Flughäfen wichtig sind, und wie gut sich die Flughäfen auf die Airlines verteilen.

```
ggplot(data = flights, mapping = aes(x = carrier, fill = origin)) +  
  geom_bar(position = "dodge")
```

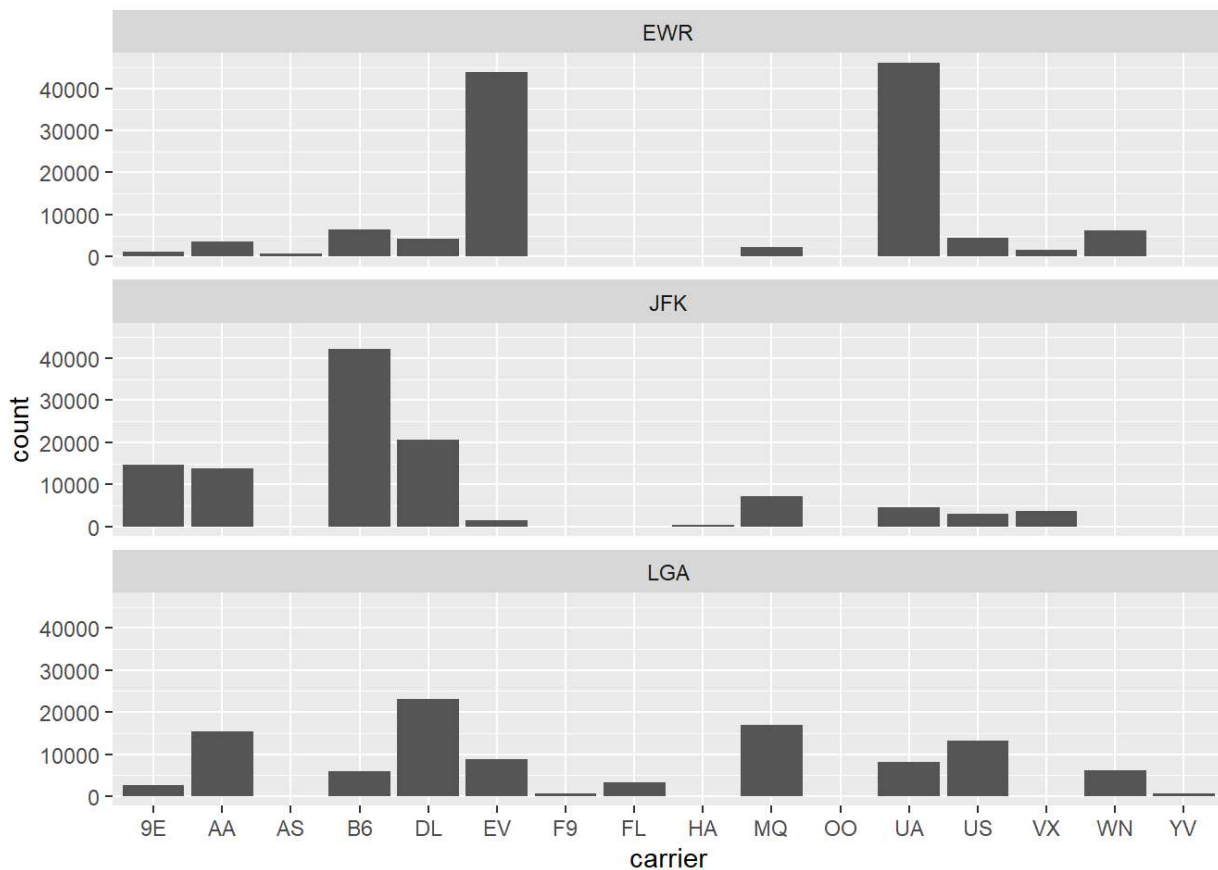


```
# --> Bars have different width if there are less than three origins per carrier.  
# Therefore:  
ggplot(data = flights, mapping = aes(x = carrier, fill = origin)) +  
  geom_bar(position = position_dodge(preserve = "single"))
```



Schließlich können wir die Balkendiagramme getrennt für die drei Flughäfen erstellen und neben- oder untereinander anzeigen lassen:

```
ggplot(data = flights, mapping = aes(x = carrier)) +
  geom_bar() +
  facet_wrap(~ origin, ncol = 1)
```



Lernkontrolle:

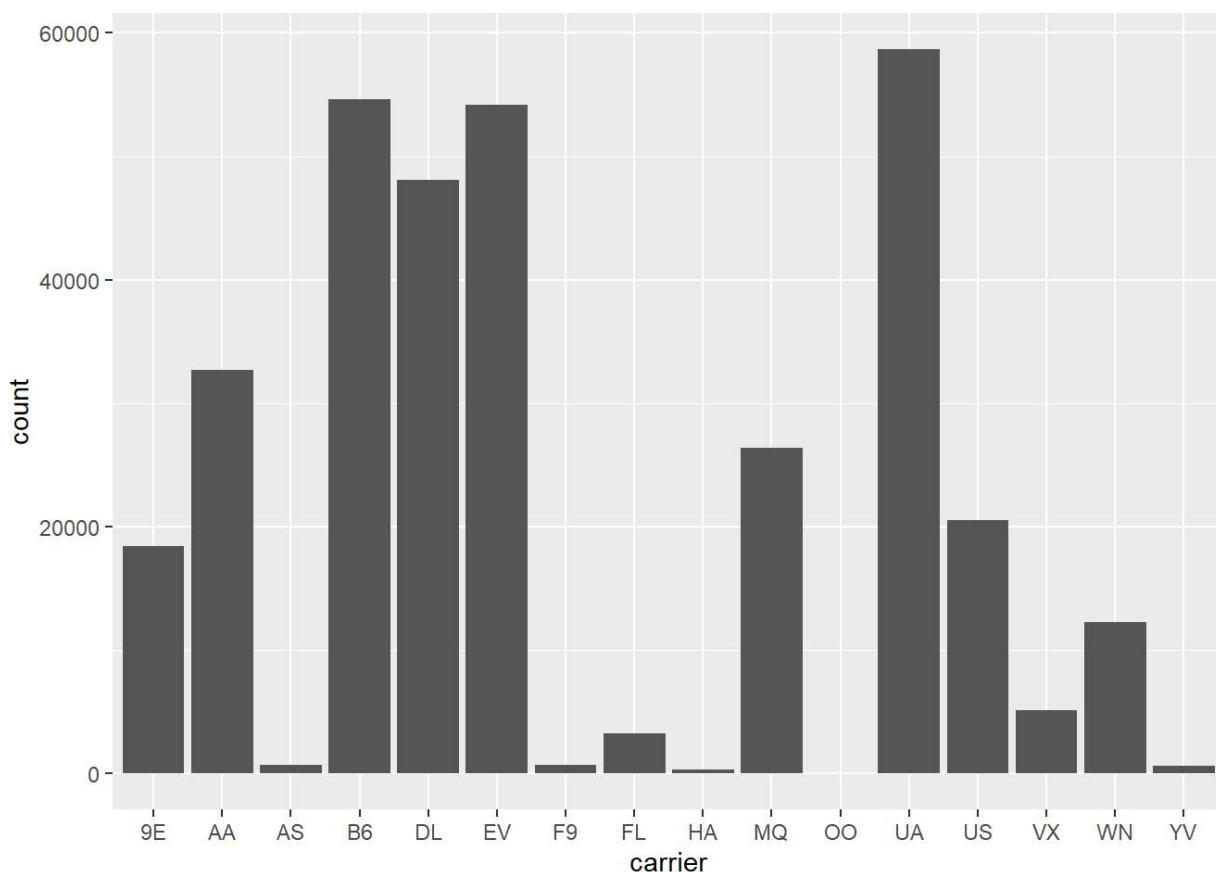
- Welche Fragen lassen sich bei der Betrachtung des gestapelten Balkendiagramms nicht so einfach beantworten?
- Was können Sie, wenn überhaupt, über die Beziehung zwischen Fluggesellschaft und Flughafen in New York City im Jahr 2013 in Bezug auf die Anzahl der abfliegenden Flüge sagen?
- Warum ist in diesem Fall ein nebeneinander angeordnetes Balkendiagramm einem gestapelten Balkendiagramm vorzuziehen?
- Welche Nachteile hat die Verwendung eines gestapelten Barplots im Allgemeinen?
- Kann das Facet-Säulendiagramm in diesem Fall einem Side-by-Side- und gestapeltem Säulendiagramm vorgezogen werden?

6.6.1 Werte umsortieren

Wir gehen nochmal einen Schritt zurück und betrachten nur die Häufigkeit von Flügen nach Fluggesellschaft.

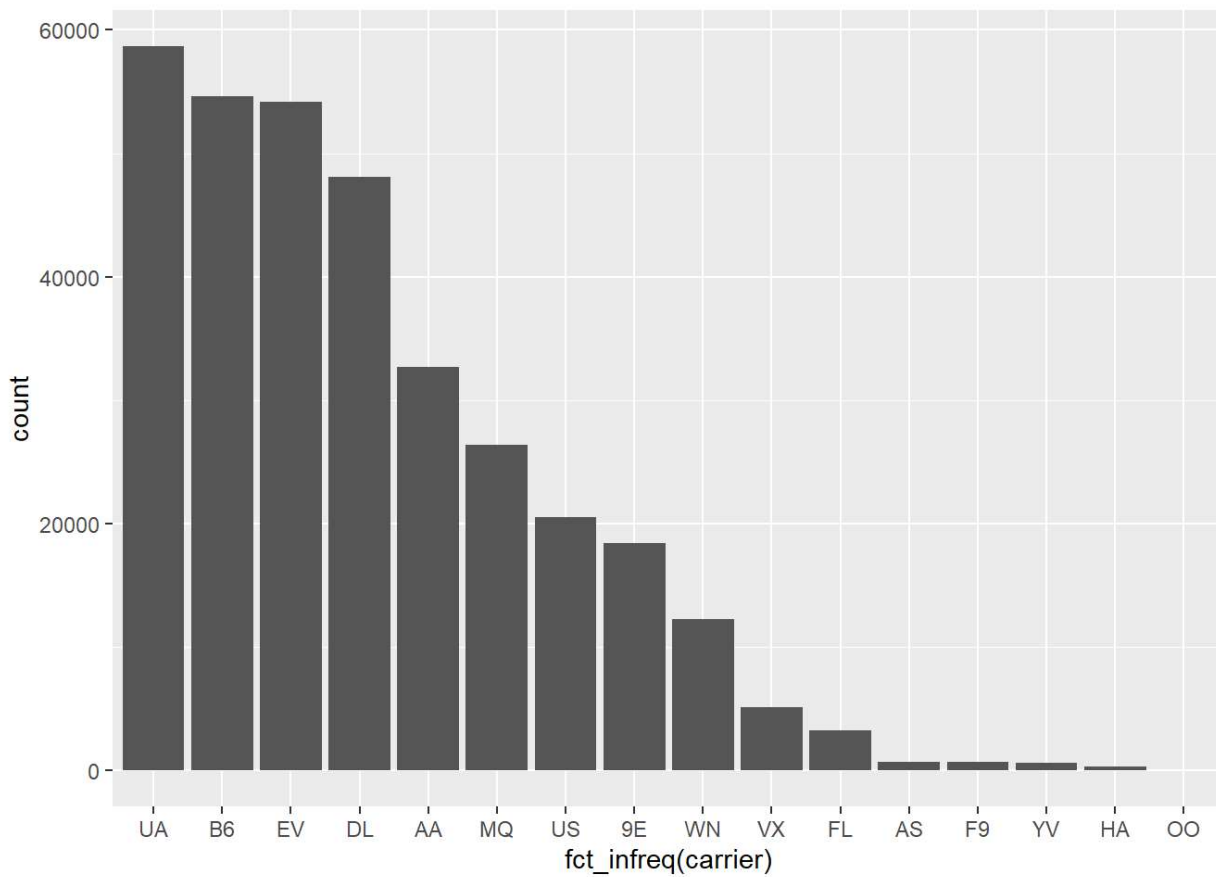
```
View(flights)

ggplot(flights, aes(x=carrier)) +
  geom_bar()
```



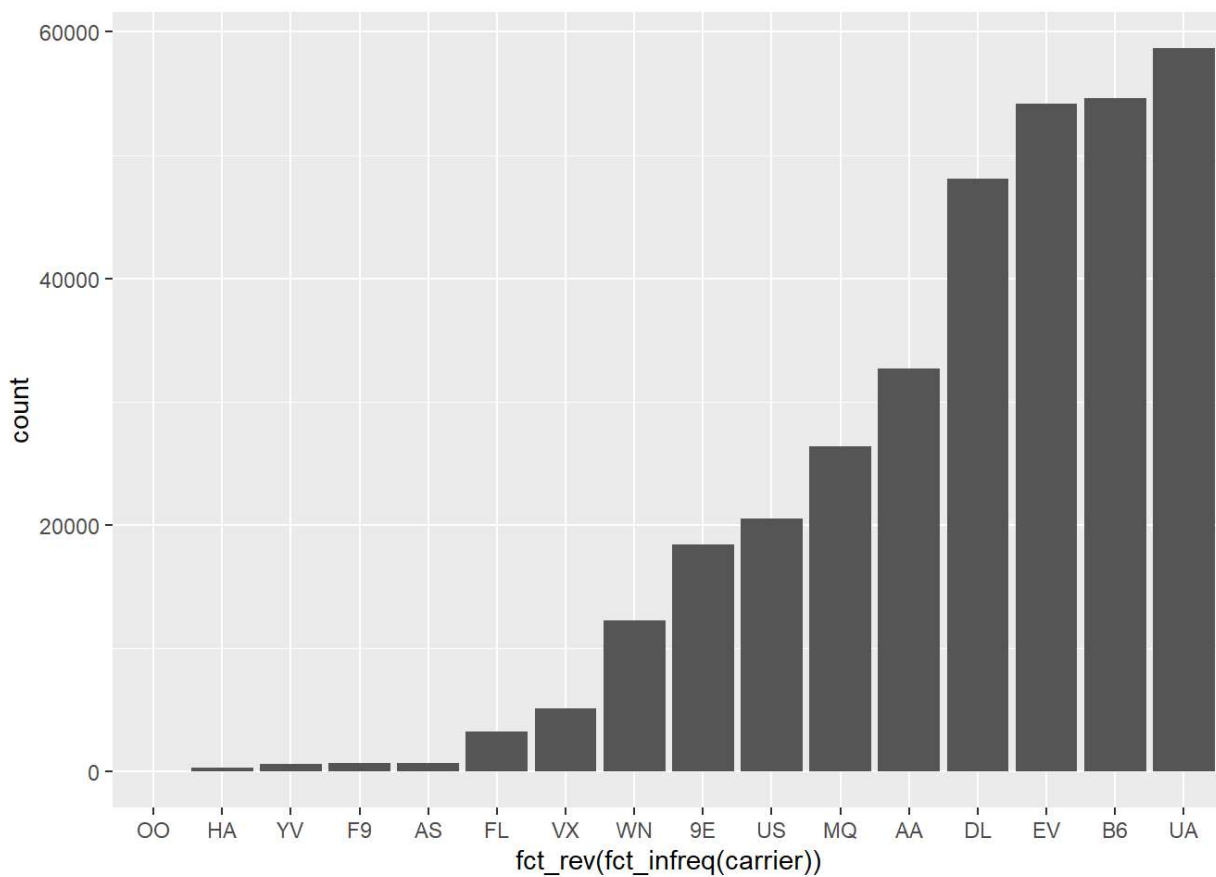
Die Grafik ist alphabetisch nach den Kürzeln der Fluggesellschaften sortiert. Das ist nicht so schön. Wir möchten lieber, dass die Werte nach ihrer Häufigkeit sortiert werden und zwar absteigend. Dies erreichen wir, indem wir `fct_infreq` ergänzen.

```
ggplot(flights, aes(x=fct_infreq(carrier))) +
  geom_bar()
```



Natürlich geht das auch aufsteigend, indem wir noch `fct_rev` voranstellen:

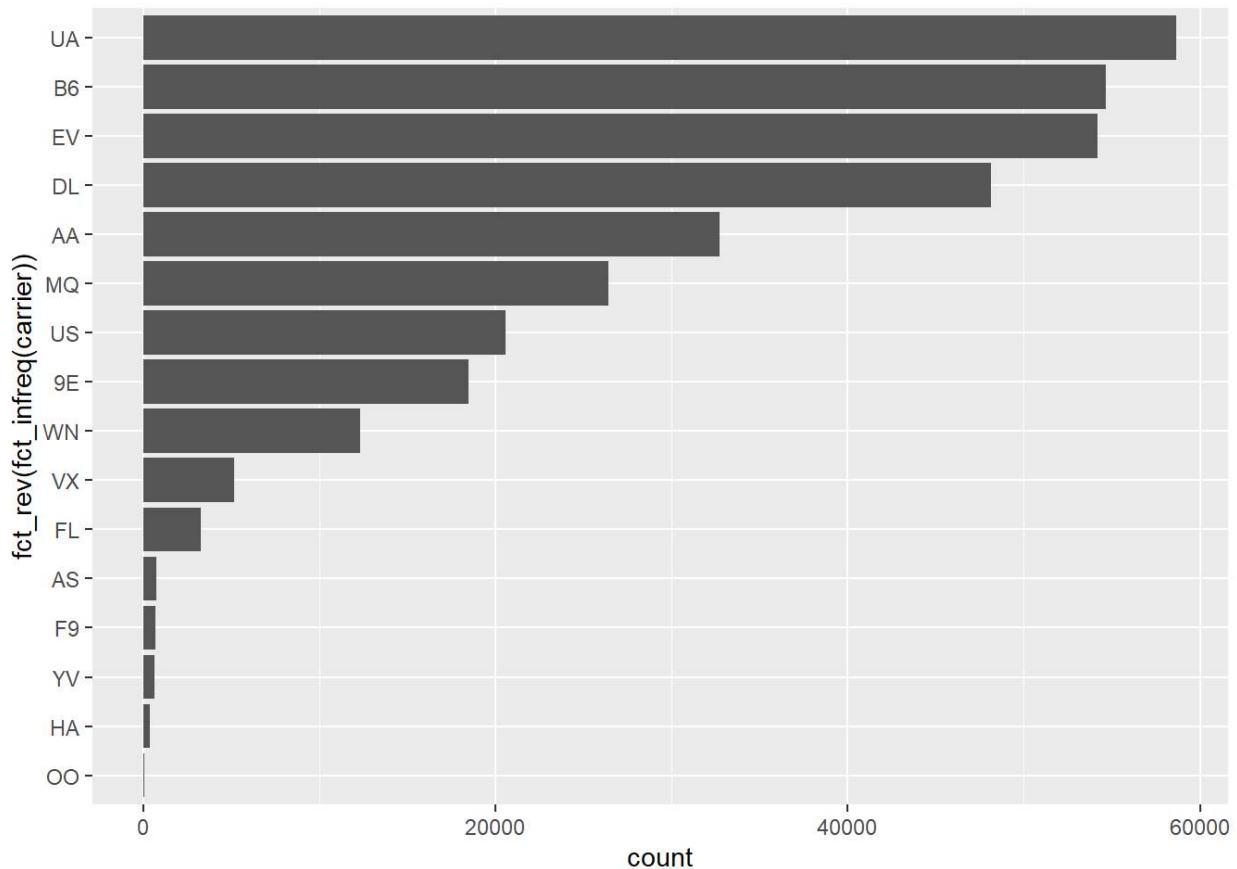
```
ggplot(flights, aes(x=fct_rev(fct_infreq(carrier)))) +  
  geom_bar()
```



Natürlich

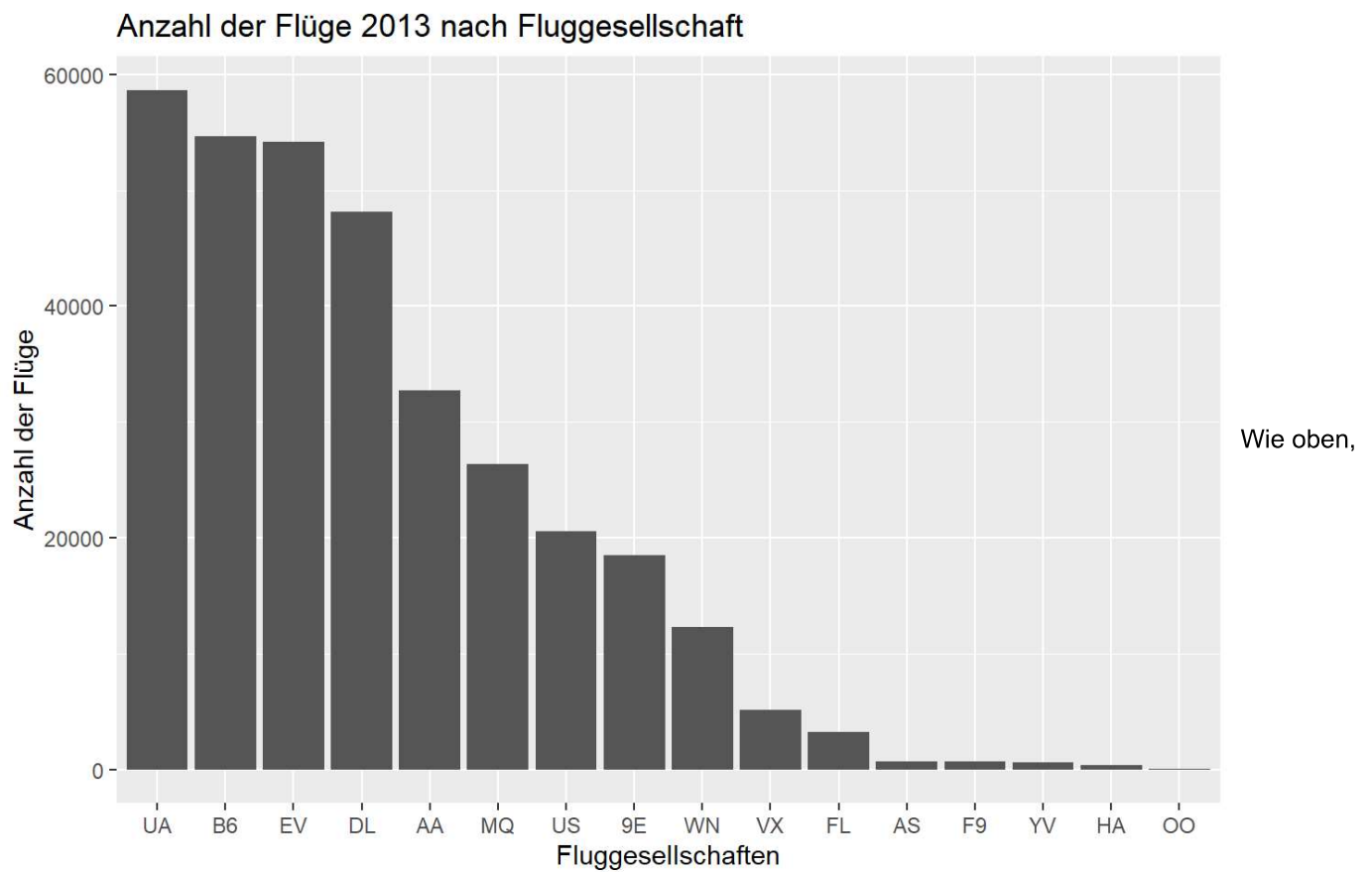
können wir das Balkendiagramm auch drehen:

```
ggplot(flights, aes(y=fct_rev(fct_infreq(carrier)))) +  
  geom_bar()
```



Leider ist es für Dritte kaum noch möglich zu erkennen, was genau eigentlich abgebildet wird. Wir sollten das Diagramm dringend beschriften!

```
ggplot(flights, aes(x=fct_infreq(carrier))) +  
  geom_bar() +  
  labs(y = "Anzahl der Flüge", x="Fluggesellschaften", title = "Anzahl der Flüge 2013 nach Fluggesellschaft")
```

können wir noch eine zweite Variable abbilden. Und wir möchten die Grafik exportieren. Dazu müssen wir die Grafik zunächst in einem Objekt abspeichern und dieses dann exportieren.

```
Bild1 <- ggplot(flights, aes(x=fct_infreq(carrier), fill=origin)) +  
  geom_bar() +  
  labs(y = "Anzahl der Flüge", x="Fluggesellschaften", title ="Anzahl der Flüge 2013 nach Fluggesellschaft und Startflughafen")  
  
ggsave(paste0(graf, "Testbild.png"), plot = Bild1, width = 10, height = 6, dpi = 300)
```